MCO-1D ORNITHOPTER

Complete Commissioning and Operating Manual

Bench Tests · TC-02 · TC-03 · Ground Station · First Flight · Revision 9

For personnel who are new to this project. Every step explained from first principles. Nothing assumed.

CONFIDENTIAL — Programme Document

TABLE OF CONTENTS

1. What This Aircraft Is and What It Does
2. Hardware Inventory — Every Part Listed
3. Electronics Wiring — What Connects to What
4. Software — What It Is and How It Works
5. Setting Up the Raspberry Pi
6. Loading the Wave Library onto the Pi
7. Setting Up the Laptop Ground Station
8. ODrive Configuration — Step by Step
9. Pixhawk Configuration — Step by Step
10. Servo Configuration and Calibration
11. TC-02 Static Thrust Test — Complete Procedure
12. TC-03 Dynamic Thrust Test — Complete Procedure
13. First Free Flight — Checklist and Procedure
14. What the Optimizer Is Doing and How to Read It
15. Emergency Procedures
16. Glossary — Every Technical Term Explained
17. Gust Response System — GR-1 through GR-8
18. New Flight Modes — Backward, Inclined Hover, Inverted Recovery
19. Software Updates and Performance
20. High-Speed Sprint and Progressive Wing Stowing
21. Autonomous Mode Discovery and Lever Evolution
22. Loop 9 — Meta-Optimizer
23. Ground Operations — Complete System
24. Ground to Air and Air to Ground Transitions
25. Touch and Go Operations
26. Emergency Lever Response
27. Unified Lever Convention — Forward Equals Power
28. Snap Roll and Gesture Mapping
29. Touchscreen Panel — Four Buttons
30. Autonomous Mode Discovery and Optimizer Overview

SECTION 1 — WHAT THIS AIRCRAFT IS AND WHAT IT DOES

1.1 The Basic Idea

The MCO-1D is an ornithopter — an aircraft that flies by flapping its wings, like a bird or a dragonfly, rather than using a propeller or jet engine. It has four pairs of wings (eight wings in total) that beat up and down continuously while in flight. The pattern in which the wings beat — how fast, how far, and with what timing between pairs — is controlled by a computer running software called the Wave Library.

Unlike a conventional aircraft where the engine produces thrust and the wings produce lift as separate functions, on the MCO-1D the flapping wings do both simultaneously. Flapping faster produces more lift and more thrust. Changing how far the wings travel on each stroke changes the aerodynamic character of the flight.

1.2 What Makes This Design Special

The wings beat in a specific pattern called a **metachronal cascade**. This means the four pairs do not all beat at exactly the same moment — each pair is slightly offset in time from the next. This is the same pattern used by dragonflies, which are among the most aerodynamically capable insects in nature. Dragonflies can hover, fly backwards, turn 180 degrees in three wingbeats, and achieve speeds up to 54 km/h. The MCO-1D is designed to replicate and extend these capabilities.

1.3 The Purpose — Search and Rescue

The MCO-1D is designed for Search and Rescue (SAR) operations. It needs to be able to: fly to a location quickly, hover or fly slowly over a target area while a camera searches, manoeuvre in confined spaces such as between buildings or in wooded terrain, and land or deploy equipment with precision. The combination of flapping-wing efficiency and the ability to fly at very slow speeds — capabilities that conventional drones cannot easily combine — makes it well suited to this role.

1.4 The Three Main Systems

Everything in this project falls into one of three categories:

- **Mechanical:** The wings, the fuselage structure, the sector wheels that drive the wing motion, the ropes and capstans, and the landing gear.
- **Electrical:** The battery, the motor controllers (called ODrives), the motors that drive the wings, the servo motors that control wing angle, the autopilot board (Pixhawk), and the load cells used for testing.
- **Software:** The Wave Library running on the Raspberry Pi, the ground station application on the laptop, and the autopilot firmware on the Pixhawk.

This manual covers all three in the order you need to work through them.

SECTION 2 — HARDWARE INVENTORY — EVERY PART LISTED

Before connecting anything, confirm you have every item in this list. If anything is missing, do not proceed — working with incomplete hardware can damage components.

2.1 Airframe Components

Wings	8 total (4 pairs). Each wing is built from carbon fibre spars and a membrane. Wings 1 and 2 are the largest (front pair), Wing 4 pair is the smallest.
Fuselage	The main body spine. All other components attach to this.
Sector wheels	8 total, one per wing. These are the disc-shaped mechanical components at each wing root that the rope wraps around to drive the flapping motion. Each has 200 degrees of total travel (±100 degrees from centre).
Ropes/Capstans	The mechanical link between the motor shaft and the sector wheel. Must be tensioned correctly.
Landing gear	Three-point wheeled undercarriage. Allows ground roll and taxi.
Wing root joints	The pivot point for each wing. Allows the wing to sweep (fold forward/aft) and change dihedral (tilt up/down).

2.2 Motors

Flap motors	8 × GoolRC 5010 360KV brushless motors. One per wing. These drive the flapping motion through the sector wheel. 360KV means the motor spins at 360 RPM per volt applied.
Motor purpose	The flap motors are the primary thrust and lift producers. They are the equivalent of the engine on a conventional aircraft.

2.3 Motor Controllers (ODrives)

An ODrive is a precision motor controller. It takes commands from the Raspberry Pi and converts them into exactly the right electrical signals to make the motor move to a specific position at a specific speed. Think of it as the interface between the computer (which speaks digital) and the motor (which speaks electrical current).

ODrive units	4 units, each controlling 2 motors (one per channel). Total: 4 ODrives × 2 channels = 8 motor channels.
Communication	CAN-FD bus. This is a high-speed digital communication standard, the same type used in modern cars. All 4 ODrives share a single cable pair running in a daisy-chain.
Power input	4S LiPo battery, approximately 14.8V. Each ODrive has its own power connection from the battery distribution board.

Key setting: vel_limit	Must be set to 150 rad/s before first flight. This is the maximum angular velocity of each motor shaft. Factory default is much lower and must be changed. Procedure in Section 8.

2.4 Servo Motors

Servo motors are small, precise positioning motors. They receive a pulse-width signal and move to a specific angle. Unlike the flap motors, servos do not spin continuously — they move to a commanded position and hold it.

Pitch servos	8 × Helidirect TORQ TCM-1001. One per wing. These rotate the wing chord to change the angle of attack — the angle at which the wing meets the air. Controls how much lift each stroke produces. Operate at 333Hz PWM signal, 7.4V supply.
Sweep servos	8 × DS3225-class servos. One per wing. These fold the wing forward or backward (sweep angle). Used to change the wing's shape for different flight modes. Operate at 50Hz PWM.
PWM signal	Pulse Width Modulation. The servo reads a signal that pulses between 1000 microseconds (full one direction) and 2000 microseconds (full other direction), with 1500 microseconds as neutral.

2.5 Flight Controller (Pixhawk)

The Pixhawk is the autopilot board. It contains an Inertial Measurement Unit (IMU) — a combination of accelerometers and gyroscopes — that measures the aircraft's attitude (which way it is pointing) and motion at all times. It also has a barometric altimeter, GPS, and communicates with the Raspberry Pi.

IMU	Measures roll (banking), pitch (nose up/down), and yaw (turning). Updates at 400Hz.
Barometer	Measures altitude by air pressure. Accurate to approximately ±3 metres.
GPS	Provides position, groundspeed, and heading. Requires clear sky view to acquire satellites.
Communication to Pi	Serial UART at 921600 baud, using MAVLink protocol.
Power	5V regulated, from the power module.

2.6 Raspberry Pi

The Raspberry Pi is the main flight computer. It runs the Wave Library software that controls everything. The Pixhawk provides sensor data; the Pi processes it and sends commands to the ODrives and servos. Think of the Pi as the pilot and the Pixhawk as the instruments.

Model	Raspberry Pi 4 (recommended) or Pi 3B+. Pi Zero 2W can work but needs USB-ethernet adapter.

Storage	MicroSD card, minimum 32GB, Class 10 or better. This holds the operating system and the Wave Library.
Power	5V, 3A minimum. Use the official Pi power supply or a quality USB-C supply.
GPIO pins	The 40-pin header on the Pi connects to the servo signal wires. Specific pin assignments are in Section 3.
Ethernet port	Used for laptop connection during testing. Direct cable to laptop — no router needed.

2.7 Load Cells (for TC-02 and TC-03)

A load cell is a precision force-measuring sensor. During the tethered thrust tests it measures exactly how much upward force the aircraft is producing. This is how we know whether the aircraft can carry its own weight.

Load cell type	HX711-based strain gauge load cell. Rated capacity must exceed the aircraft weight (34N minimum — recommend 50N or 100N rated cell).
HX711 amplifier	The HX711 is a small circuit board that amplifies the tiny signal from the load cell to a level the Pi can read.
Connection to Pi	Two GPIO pins: DT (data) on GPIO 26, SCK (clock) on GPIO 7. Four wires total (DT, SCK, VCC 3.3V, GND).
Mounting	The aircraft hangs or sits on the test rig. The load cell is between the aircraft and the fixed structure. As the wings produce lift, the force reading increases.

2.8 Battery

Type	4S LiPo (Lithium Polymer). 4S means four cells in series.
Voltage	Nominal 14.8V. Fully charged: 16.8V. Do not discharge below 14.0V (3.5V per cell).
Capacity	Recommend 4000-6000mAh for useful flight time. Larger = heavier = less performance.
Connector	XT60. The battery connects to the power distribution board which then feeds the ODrives, Pi, and Pixhawk separately.
WARNING	LiPo batteries are potentially dangerous if punctured, overcharged, or discharged too far. Always use a proper LiPo charger. Never leave charging unattended. Store in a LiPo-safe bag.

2.9 Laptop (for Ground Station)

Any laptop	Mac, Linux, or Windows. Must have Python 3.8 or later installed.

Ethernet port	Needed for direct cable connection to Pi. If laptop has no ethernet port, a USB-to-ethernet adapter works fine.
Software needed	Only Python 3.8+. The ground station application (mco_ground_station.py) uses only standard Python libraries — nothing to install.

SECTION 3 — ELECTRONICS WIRING — WHAT CONNECTS TO WHAT

■ BEFORE CONNECTING ANYTHING: Remove the battery. Work with power disconnected at all times until Section 10 (calibration) specifically instructs you to connect power. Connecting power to wired electronics can cause permanent damage.

3.1 Overview — The Signal Flow

Understanding signal flow makes the wiring logical. Here is the path a command takes from pilot input to physical wing movement:

- Pilot moves lever on transmitter
- Transmitter sends radio signal to receiver
- Receiver sends signal to Pixhawk
- Pixhawk sends sensor data to Raspberry Pi (via serial)
- Raspberry Pi runs Wave Library, computes wing commands
- Pi sends position commands to ODrives (via CAN-FD)
- Pi sends angle commands to servos (via GPIO PWM)
- ODrives drive flap motors
- Servos move pitch and sweep angles
- Wings move — aircraft flies

3.2 CAN-FD Bus — Connecting the ODrives

CAN-FD (Controller Area Network, Flexible Data rate) is a two-wire communication bus. All four ODrives share a single pair of wires (CAN-H and CAN-L) running in a daisy-chain from the Pi to ODrive 1, then to ODrive 2, then ODrive 3, then ODrive 4.

Wires needed	Twisted pair, 24AWG minimum. Use CAT5 ethernet cable — it is already twisted.
Pi CAN interface	MCP2515 CAN hat for Raspberry Pi, or a USB-to-CAN adapter (PCAN-USB or similar).
ODrive CAN pins	Labelled CAN-H and CAN-L on each ODrive. Connect H to H and L to L throughout the chain.
Termination resistors	120 ohm resistors at BOTH ends of the CAN chain. The Pi end and the last ODrive end. Without these the signal reflects and communication fails. Most CAN hats have a built-in termination jumper.
ODrive CAN IDs	Each ODrive must have a unique CAN node ID. Set via ODrive Tool software before installation. IDs 0, 1, 2, 3 for the four units.
CAN speed	Set to 1000000 (1 Mbit/s) on all devices including the Pi CAN interface.

■ CAN-FD is robust and noise-resistant. If communication seems intermittent, check the termination resistors first — this is the most common cause of CAN problems.

3.3 Servo Connections to Raspberry Pi GPIO

The Raspberry Pi communicates with servos via GPIO (General Purpose Input Output) pins. Each servo needs one signal wire connected to a specific GPIO pin on the Pi. All servos also need power (7.4V for pitch servos, 5-6V for sweep servos) and ground — these come from a separate BEC (Battery Eliminator Circuit), not from the Pi.

■ **Never connect servo power (7.4V) to the Pi GPIO pins. GPIO pins are 3.3V signal only. Power goes to the servo rail separately.**

Pitch Servo Pin Assignments (GPIO BCM numbering):

Wing 1 Port (W1p)	GPIO 18
Wing 1 Starboard (W1s)	GPIO 23
Wing 2 Port (W2p)	GPIO 24
Wing 2 Starboard (W2s)	GPIO 25
Wing 3 Port (W3p)	GPIO 12
Wing 3 Starboard (W3s)	GPIO 16
Wing 4 Port (W4p)	GPIO 20
Wing 4 Starboard (W4s)	GPIO 21

Sweep Servo Pin Assignments:

Wing 1 Port sweep	GPIO 4
Wing 1 Starboard sweep	GPIO 17
Wing 2 Port sweep	GPIO 27
Wing 2 Starboard sweep	GPIO 22
Wing 3 Port sweep	GPIO 5
Wing 3 Starboard sweep	GPIO 6
Wing 4 Port sweep	GPIO 13
Wing 4 Starboard sweep	GPIO 19

■ BCM (Broadcom) numbering is used — NOT physical pin numbers. BCM 18 is physical pin 12. Always verify with a Pi GPIO pinout diagram before connecting.

3.4 Load Cell (HX711) to Pi

HX711 DT (Data)	Pi GPIO 26 (physical pin 37)

HX711 SCK (Clock)	Pi GPIO 7 (physical pin 26)
HX711 VCC	Pi 3.3V (physical pin 1 or 17)
HX711 GND	Pi GND (physical pin 9, 25, or 39)
Load cell wires	4-wire load cell: Red=Excitation+, Black=Excitation-, White=Signal+, Green=Signal-. Connect to matching HX711 terminals.

3.5 Pixhawk to Raspberry Pi

Connection type	Serial UART (TELEM2 port on Pixhawk)
Pi side	GPIO 14 (TXD) and GPIO 15 (RXD) — the Pi's hardware serial port, or a USB serial adapter
Pixhawk TELEM2	6-pin JST-GH connector. Pin 2 = TX (connects to Pi RX). Pin 3 = RX (connects to Pi TX). Pin 6 = GND (connects to Pi GND). TX and RX always cross: Pixhawk TX → Pi RX.
Baud rate	921600 baud. Set in both Mission Planner (Pixhawk) and the Wave Library.
Power	Do NOT power the Pi from the Pixhawk or vice versa. Each has its own regulated supply.

3.6 Laptop to Raspberry Pi (Ground Station)

Cable	Standard ethernet cable (Cat5e or Cat6), any length. No crossover cable needed — modern network adapters auto-negotiate.
Pi IP address	192.168.7.1 (set as static)
Laptop IP address	192.168.7.2 (set as static)
Port	5760 (TCP). The ground station server listens on this port.
No router needed	This is a direct point-to-point connection. No WiFi, no router, no internet required.

■ A direct ethernet cable is strongly preferred over WiFi for TC-02 and TC-03. Motor electrical noise can interfere with WiFi and cause dropouts at the worst possible moment.

3.7 Power Distribution

The battery connects to a power distribution board (PDB). From the PDB:

- ODrives: direct 4S LiPo voltage (14.8V). Each ODrive has its own XT30 or XT60 connector.
- Pitch servos: 7.4V regulated BEC. The Helidirect TCM-1001 servos require 7.4V.
- Sweep servos: 5-6V regulated BEC.

- Raspberry Pi: 5V, 3A regulated supply. USB-C connection. Use a quality regulated supply — the Pi is sensitive to voltage fluctuations.
- Pixhawk: 5V from the Pixhawk power module, which also measures battery voltage and current.

■ Connect the battery LAST after all other wiring is complete and verified. Disconnect the battery FIRST before any wiring changes.

SECTION 4 — SOFTWARE — WHAT IT IS AND HOW IT WORKS

4.1 The Three Software Components

There are three pieces of software in this system. They run on different computers and do different things.

Wave Library (mco_wave_library.py)	Runs on the Raspberry Pi. This is the main brain — 24,000+ lines of Python that control every aspect of flight. Reads sensors, computes wing commands, runs the optimizers, manages all safety systems. This is what we have been building throughout this project.
Ground Station (mco_ground_station.py)	Runs on the laptop. A terminal application that connects to the Pi over ethernet and shows you live data from the aircraft. Lets you send commands to the Pi, monitor test progress, and control the optimizer.
Pixhawk Firmware (ArduPilot or PX4)	Runs on the Pixhawk autopilot board. Pre-built firmware — you do not write this code. It provides sensor data (IMU, GPS, barometer) to the Pi via MAVLink protocol.

4.2 What the Wave Library Does

The Wave Library runs a continuous loop — called a "tick" — 50 times per second (every 20 milliseconds). Each tick it:

- Reads all sensor data from the Pixhawk (attitude, airspeed, altitude, GPS position)
- Reads load cell force measurement (during tests)
- Reads pilot control inputs (lever, stick, rudder)
- Runs safety checks (stall speed, never-exceed speed, ground clearance)
- Computes the optimal wing configuration for the current flight condition
- Runs the adaptive optimizers (Loops 1-7) that improve performance over time
- Sends position commands to the ODrives
- Sends angle commands to the pitch and sweep servos
- Sends telemetry to the ground station laptop

This cycle repeats 50 times every second, continuously, for the entire duration of flight.

4.3 The Optimizer System

The Wave Library contains seven adaptive optimizer loops (Loop 1 through Loop 7). These are learning systems that observe how the aircraft is performing and make small adjustments to improve it over time. They do not change anything dramatically — they make tiny, measured changes and observe the result, like a pilot making a series of small trim adjustments. The optimizers improve the aircraft's performance with every flight.

Loop 1 (Stroke level)	Runs every wing stroke. Adjusts the angle of attack (AoA) on the downstroke and upstroke to maximise lift efficiency.
Loop 2 (Cascade phase)	Runs every 2 seconds. Adjusts the timing offset between wing pairs to recover energy from the wake of the front pair (like a coaxial helicopter recovering swirl).
Loop 3 (Surge)	Runs every 5 seconds. Adjusts how the force is distributed between front and rear wing pairs.
Loop 4 (Banked config)	Runs every 12 seconds during banked flight. Adjusts the split-stroke geometry for knife-edge and banked turns.
Loop 5 (Yaw modality)	Runs every 15 seconds. Discovers the best combination of crab, dart, split-stroke, and proverse yaw for efficient turning.
Loop 6 (Lever map)	Runs every 30 seconds. Evaluates whether any discovered wing configurations outperform the current lever zone assignments and proposes remaps.
Loop 7 (Lever feel)	Runs every 45 seconds. Adjusts the sensitivity of the wave lever to minimise how often the pilot needs to make a correction — making the aircraft feel predictable and smooth.

SECTION 5 — SETTING UP THE RASPBERRY PI

5.1 Install the Operating System

The Raspberry Pi needs a Linux operating system. We use Raspberry Pi OS (formerly called Raspbian). This is a free download. Follow these steps exactly.

Step 1 — Download Raspberry Pi Imager:

- Go to: https://www.raspberrypi.com/software/
- Download the Imager for your laptop's operating system (Windows, Mac, or Linux).
- Install and open the Imager application.

Step 2 — Flash the MicroSD card:

- Insert the MicroSD card into your laptop (use an adapter if needed).
- In Raspberry Pi Imager: Choose Device: Raspberry Pi 4 (or your model). Choose OS: Raspberry Pi OS (64-bit). Choose Storage: your MicroSD card.
- Click the gear icon (settings) before writing. Set hostname: mco-1d. Enable SSH. Set username: pi. Set password: choose something memorable. Configure WiFi if you want it (optional).
- Click Write. Wait for it to complete (5-15 minutes).
- Remove the MicroSD and insert it into the Pi.

Step 3 — First boot:

- Connect the Pi to a monitor, keyboard, and mouse (only needed for first setup).
- Connect power. The Pi will boot in 1-2 minutes.
- If you set up SSH, you can also connect from your laptop using: ssh pi@mco-1d.local

5.2 Configure Static IP Address

The Pi needs a fixed (static) IP address so the laptop can always find it at the same address regardless of which network it is on.

Open a terminal on the Pi (or SSH in) and run:

```
sudo nano /etc/dhcpcd.conf
```

Scroll to the bottom of the file and add these lines:

```
interface eth0 static ip_address=192.168.7.1/24 static routers=192.168.7.1 static
domain_name_servers=8.8.8.8
```

Save with Ctrl+X, then Y, then Enter. Reboot:

```
sudo reboot
```

After reboot, verify with:

```
ip addr show eth0
```

You should see 192.168.7.1 in the output.

5.3 Install Required Python Packages

The Wave Library needs a few Python packages. Run these commands in the Pi terminal:

```
sudo apt update sudo apt install python3-pip python3-rpi.gpio -y pip3 install pigpio smbus2 numpy --break-system-packages
```

Start the pigpio daemon (needed for servo PWM):

```
sudo systemctl enable pigpiod sudo systemctl start pigpiod
```

Verify pigpio is running:

```
sudo systemctl status pigpiod
```

You should see "active (running)" in green.

5.4 Install Python CAN Library

The Wave Library communicates with the ODrives over CAN-FD. Install the CAN library:

```
pip3 install python-can odrive --break-system-packages
```

Enable the SPI interface (needed for the MCP2515 CAN hat):

```
sudo raspi-config
```

Navigate to: Interface Options → SPI → Enable. Then reboot.

SECTION 6 — LOADING THE WAVE LIBRARY ONTO THE PI

6.1 Transfer the Files

You need to copy two files from your development computer to the Raspberry Pi:

- mco_wave_library.py — the main flight software
- mco_ground_station.py — the ground station client (this actually runs on the laptop, but keep a copy on the Pi too)

Method 1 — Using SCP (secure copy) over the network:

```
# Run this on your laptop, not the Pi scp mco_wave_library.py pi@192.168.7.1:~/ scp
mco_ground_station.py pi@192.168.7.1:~/
```

Method 2 — Using a USB drive:

- Copy files to a USB drive on your laptop.
- Plug the USB drive into the Pi.
- On the Pi: sudo cp /media/pi/USBDRIVE/mco_wave_library.py ~/

6.2 Verify the Library Loads

SSH into the Pi and run:

```
cd ~ python3 -c "import mco_wave_library as w; print('Library OK')"
```

If you see "Library OK" the file loaded without errors. If you see error messages, check that all required packages from Section 5.3 are installed.

6.3 How to Interact with the Library

The Wave Library is a Python module. You interact with it by writing short Python scripts that import it, or by typing commands in the Python interactive interpreter. During TC-02 and TC-03 you will use pre-written test scripts.

To start an interactive session on the Pi:

```
python3 >>> import mco_wave_library as w >>> w.gs_start() # starts ground station server
>>> w.gnd_clear_status() # shows ground clearance status >>> exit() # to quit
```

The most important functions during testing are:

w.gs_start()	Start the ground station TCP server. Call this first — it lets the laptop connect.
w.aero_cal_status()	Show aerodynamic calibration status. How many observations have been collected.
w.lm6_zone_report()	Show the current lever zone map and any pending remap candidates.

w.violent_manoeuvre_status()	Show which safety systems are currently active.
w.l7_status()	Show the lever feel optimizer — how smooth the lever feels and how it is being tuned.

6.4 The TC-02 Test Script

Create a file on the Pi called run_tc02.py:

```
import mco_wave_library as w # Start ground station so laptop can monitor w.gs_start() # Connect hardware (adjust these for your setup) import pigpio pi = pigpio.pi() # Run the static thrust test # Parameters: pigpio instance, ODrive axes list, # load cell reader, output CSV path results = w.tc02_static_thrust_test( pi_handle=pi, odrive_axes=None, # replace with actual axes loadcell=None, # replace with HX711 reader ) # Feed results into calibration system w.tc02_feed_calibration(results['results']) w.aero_cal_status() pi.stop()
```

Run it with:

```
python3 run_tc02.py
```

SECTION 7 — SETTING UP THE LAPTOP GROUND STATION

7.1 What the Ground Station Shows

The ground station application (mco_ground_station.py) runs on your laptop and shows a live display of everything happening on the Pi. It refreshes automatically every half second. You can also type commands to send to the Pi.

The display is divided into panels:

- **Flight State:** airspeed, height, specific excess power (Ps — positive means climbing, negative means descending), bank angle, flap frequency, lever position. Also shows which systems are currently active.
- **TC-02/TC-03 Data:** each test point displayed the instant the load cell reading completes. Shows lift force in Newtons, CT (thrust coefficient), and whether T/W (thrust-to-weight) exceeds 1.0.
- **Lever Zone Map:** current zone assignments and any modes the optimizer is proposing to add.
- **Command prompt:** type commands here to control the Pi remotely.

7.2 Configure the Laptop Static IP

The laptop needs a static IP on the same subnet as the Pi (192.168.7.x).

On macOS:

- System Settings → Network
- Select your ethernet adapter (or USB ethernet)
- Click Details
- TCP/IP tab → Configure IPv4: Manually
- IP Address: 192.168.7.2
- Subnet Mask: 255.255.255.0
- Router: (leave blank)
- Click OK and Apply

On Linux:

```
sudo ip addr add 192.168.7.2/24 dev eth0 # Replace eth0 with your actual interface name #
Find it with: ip link show
```

On Windows:

- Control Panel → Network and Internet → Network Connections
- Right-click your ethernet adapter → Properties
- Select "Internet Protocol Version 4 (TCP/IPv4)" → Properties
- Select "Use the following IP address"
- IP: 192.168.7.2 Subnet: 255.255.255.0 Gateway: (blank)
- Click OK

7.3 Verify the Connection

With the ethernet cable connected and both IPs set, test from the laptop:

```
ping 192.168.7.1
```

You should see responses like "64 bytes from 192.168.7.1". If you see "Request timeout" or "No route to host", check: cable connected, Pi is on, IP addresses correct.

7.4 Install Python on the Laptop

Check if Python 3.8+ is already installed:

```
python3 --version
```

If you see Python 3.8 or higher, you are ready. If not, or if python3 is not found:

- Mac: Install from https://www.python.org/downloads/ or use Homebrew: brew install python3
- Linux: sudo apt install python3
- Windows: Download from https://www.python.org/downloads/ and run the installer. Check "Add to PATH".

7.5 Run the Ground Station

Copy mco_ground_station.py to your laptop. Then, with the Pi running and gs_start() called on the Pi:

```
python3 mco_ground_station.py 192.168.7.1
```

You will see "Connected." and the display will appear. The command prompt shows "mco>" where you type commands.

Essential commands to know:

status	Full telemetry snapshot from the Pi
aero	Aerodynamic calibration detail — shows what has been measured so far
zones	Lever zone map — shows which wing modes are assigned to which lever positions
freeze	Freeze all optimizer loops — use before changing any settings manually
restore	Unfreeze optimizer loops after changes
set KEY value	Change a setting on the Pi. Example: set WFAOA_GAIN 0.8 set LM6_AUTO_REMAP true
remap ZONE	Confirm a pending lever zone remap. Example: remap SPRINT
abort	Stop a TC-02 or TC-03 test immediately
help	Show all available commands

q	Quit the ground station

SECTION 8 — ODRIVE CONFIGURATION — STEP BY STEP

■ This section requires ODrive Tool software on your laptop and a USB connection to each ODrive one at a time. Do NOT connect the ODrives to the CAN bus until they are individually configured.

8.1 Install ODrive Tool on Laptop

```
pip3 install odrive
```

Verify:

```
odrivetool --version
```

8.2 Connect One ODrive at a Time

Take the first ODrive. Connect only:

- USB cable from ODrive to laptop
- Power supply to ODrive (use bench supply set to 14.8V, limit 2A for configuration — NOT the main battery)
- One motor connected (motor 0 or 1)

Do NOT connect the CAN bus yet.

```
odrivetool
```

You will see "Connected to ODrive..." and a prompt showing the firmware version.

8.3 Configure Each ODrive

In the ODrive Tool prompt, type these commands one by one. Press Enter after each.

Set CAN node ID (use 0 for first ODrive, 1 for second, 2 for third, 3 for fourth):

```
odrv0.can.config.node_id = 0 odrv0.can.config.baud_rate = 1000000
```

Configure motor (do this for both axis0 and axis1):

```
odrv0.axis0.motor.config.pole_pairs = 7
odrv0.axis0.motor.config.resistance_calib_max_voltage = 4
odrv0.axis0.motor.config.requested_current_range = 25
odrv0.axis0.motor.config.current_control_bandwidth = 100
odrv0.axis0.motor.config.motor_type = MOTOR_TYPE_HIGH_CURRENT
```

Set velocity limit — THIS IS CRITICAL:

```
odrv0.axis0.controller.config.vel_limit = 150 odrv0.axis1.controller.config.vel_limit =
150
```

■ The vel_limit MUST be set to 150 rad/s. The factory default is much lower and will prevent the wings from reaching the necessary flapping frequency. Without this change the aircraft cannot generate enough thrust to fly.

Set control mode to position control:

```
odrv0.axis0.controller.config.control_mode = CONTROL_MODE_POSITION_CONTROL
odrv0.axis0.controller.config.input_mode = INPUT_MODE_POS_FILTER
odrv0.axis1.controller.config.control_mode = CONTROL_MODE_POSITION_CONTROL
odrv0.axis1.controller.config.input_mode = INPUT_MODE_POS_FILTER
```

Configure encoder (hall effect sensors):

```
odrv0.axis0.encoder.config.mode = ENCODER_MODE_HALL odrv0.axis0.encoder.config.cpr = 42
odrv0.axis1.encoder.config.mode = ENCODER_MODE_HALL odrv0.axis1.encoder.config.cpr = 42
```

Save and reboot:

```
odrv0.save_configuration() odrv0.reboot()
```

Repeat for all four ODrives, changing the node_id to 1, 2, 3 for each subsequent unit.

8.4 Calibrate Each Motor

With one ODrive powered (bench supply) and one motor connected, run motor calibration:

```
odrv0.axis0.requested_state = AXIS_STATE_MOTOR_CALIBRATION
```

The motor will beep and make a short movement. Wait until it stops. Check for errors:

```
odrv0.axis0.motor.error
```

If the result is 0, calibration succeeded. If non-zero, check motor connections and power supply.

Then calibrate the encoder:

```
odrv0.axis0.requested_state = AXIS_STATE_ENCODER_HALL_POLARITY_CALIBRATION
```

Wait for completion, then:

```
odrv0.axis0.requested_state = AXIS_STATE_ENCODER_OFFSET_CALIBRATION
```

Save the calibration:

```
odrv0.axis0.config.startup_encoder_offset_calibration = True odrv0.save_configuration()
```

■ The calibration only needs to be done once per motor. If you replace a motor, repeat calibration for that channel.

SECTION 9 — PIXHAWK CONFIGURATION — STEP BY STEP

9.1 Install Mission Planner on Laptop

Mission Planner is the configuration software for the Pixhawk. Download from:

```
https://ardupilot.org/planner/docs/mission-planner-installation.html
```

Alternatively, use QGroundControl from:

```
https://docs.qgroundcontrol.com/master/en/getting_started/download_and_install.html
```

Either works. This manual uses Mission Planner examples.

9.2 Flash ArduPilot Firmware

- Connect Pixhawk to laptop via USB.
- Open Mission Planner.
- Go to Setup → Install Firmware.
- Select your vehicle type. For the MCO-1D select "ArduPlane" — closest to our fixed-wing-like control structure.
- Click the appropriate firmware version and wait for installation to complete.
- The Pixhawk will reboot with new firmware.

9.3 Key Parameter Settings

In Mission Planner, go to Config → Full Parameter List. Set these parameters:

SERIAL2_BAUD	921 (= 921600 baud — connection to Pi)
SERIAL2_PROTOCOL	2 (MAVLink 2)
ARSPD_TYPE	1 (analog airspeed sensor) or 0 if using pitot tube with digital output
COMPASS_ENABLE	1 (enable magnetometer)
INS_GYRO_FILTER	20 (gyroscope filter)
AHRS_EKF_TYPE	3 (EKF3 — best for this application)

After setting parameters, click Write Params to save to the Pixhawk.

9.4 Sensor Calibration

In Mission Planner, go to Setup → Mandatory Hardware:

- **Accelerometer Calibration:** Follow the on-screen instructions to hold the Pixhawk in 6 orientations. This calibrates the accelerometer so the IMU knows which way is up.

- **Compass Calibration:** Click Start, rotate the Pixhawk through all orientations slowly. Complete when the progress bar reaches 100%.
- **Radio Calibration:** If using an RC transmitter, move all sticks to their extremes while watching the bar graphs to set the minimum and maximum values.
- **Level Calibration:** Place the Pixhawk level on a flat surface and click Calibrate Level.

SECTION 10 — SERVO CONFIGURATION AND CALIBRATION

10.1 What Calibration Achieves

Servo calibration tells the Wave Library exactly how to convert a commanded angle (in degrees) into a pulse-width signal (in microseconds) for each specific servo. Because servos vary slightly from unit to unit, each of the 16 servos (8 pitch, 8 sweep) must be calibrated individually.

10.2 Pitch Servo Calibration

With the Pi running and servos powered (7.4V BEC), run the calibration function. SSH into the Pi and:

```
python3 >>> import mco_wave_library as w >>> import pigpio >>> pi = pigpio.pi() >>> #
Calibrate wing 1 port pitch servo (index 0) >>> w.pitch_servo_calibrate(pi, wing_index=0)
>>> # Repeat for indices 1 through 7
```

The calibration procedure for each servo:

- The servo moves to the neutral position (1500 microseconds). Observe that the wing is at 0 degrees angle of attack (chord horizontal, aligned with the wing span).
- The function then commands known angles and asks you to confirm or adjust until the physical angle matches the commanded angle.
- Results are stored in the library automatically.

10.3 Direction Verification

After calibration, run the direction verification:

```
>>> w.direction_verify(pi)
```

This commands each wing to move in a specific direction and asks you to confirm it moved the right way. If a wing moves backwards, the servo direction for that wing is reversed in the library automatically.

10.4 Startup Self-Check

Run the startup check to verify all systems:

```
>>> w.mco_startup_check(pi) # This checks all servos, CAN communications, # load cell (if
connected), and sensor readings. # Fix any reported errors before proceeding.
```

SECTION 11 — TC-02 STATIC THRUST TEST — COMPLETE PROCEDURE

11.1 What TC-02 Measures

TC-02 (Test Condition 02) is the static thrust test. "Static" means the aircraft is not moving — it is attached to a test rig and the wings flap while we measure the upward force they produce. This answers the fundamental question: can this aircraft carry its own weight?

The key number is CT — the Thrust Coefficient. This is a dimensionless number that characterises how efficiently the wings produce force. If CT multiplied by the dynamic conditions equals a force greater than the aircraft weight (34N), the aircraft can hover. If not, it needs forward speed to fly.

11.2 Test Rig Setup

Before any electronics are connected:

- Mount the aircraft rigidly to a sturdy test stand. The aircraft must not move during the test — any movement will corrupt the force readings.
- Mount the load cell between the aircraft and the test stand. The cell should be in line with the expected thrust direction (vertically, below the aircraft, so upward thrust compresses or stretches the cell depending on mounting).
- Ensure the wings have clear space to flap — at least 1 metre clearance in all directions.
- Keep all cables (power, CAN, servo signal) clear of the wing paths.

■ During TC-02 the wings flap at up to 3 Hz with 96 degrees of amplitude. Keep hands, tools, and other objects away from the wing paths at all times when the test is running.

11.3 Electronics Connection for TC-02

- Connect load cell to HX711 module.
- Connect HX711 to Pi GPIO (Section 3.4).
- Connect ODrives to CAN bus.
- Connect all 8 pitch servos to GPIO (Section 3.3).
- Connect servo power (7.4V BEC).
- Connect ethernet cable: Pi to laptop.
- Connect main battery LAST.

11.4 Starting the Test

Step 1 — Start ground station on Pi:

```
ssh pi@192.168.7.1 python3 >>> import mco_wave_library as w >>> w.gs_start()
```

Step 2 — Connect laptop ground station:

```
# On laptop python3 mco_ground_station.py 192.168.7.1
```

You should see "Connected" and the live display appears.

Step 3 — Run startup check:

```
# On Pi >>> w.mco_startup_check(pi) # Fix any errors before continuing
```

Step 4 — Run TC-02:

```
>>> results = w.tc02_static_thrust_test( ... pi_handle=pi, ... odrive_axes=axes, # your
ODrive axis list ... loadcell=lc, # your HX711 reader object ... )
```

The test runs automatically. On the laptop you will see each test point appear as it completes — frequency, amplitude, cascade phase, lift force, and CT.

11.5 What to Watch on the Laptop During TC-02

On the ground station display, watch the TC-02 Data panel. For each test point you will see:

Lift N	Force in Newtons. Aircraft weight is 33.8N. If any point shows lift > 33.8N, the aircraft can generate enough thrust to support itself.
CT	Thrust coefficient. Higher is better. Values above 0.20 indicate good thrust performance.
T/W	Thrust-to-weight ratio. Green tick = above 1.0 = more thrust than weight. This is the goal.
Profile	Which wing pitch profile was used — slow (32° AoA), cruise (22°), or climb (28°).

11.6 After TC-02 Completes

The results are automatically fed into the aerodynamic calibration system. Check the updated model:

```
# On laptop ground station mco> aero
```

You will see the updated values for CL_MAX, peak AoA, efficiency AoA, and Vs (stall speed). These values now reflect what the actual aircraft produces, not just what the aerodynamic models predicted.

The TC-02 sweep test also runs automatically after the main test — this tests different wing sweep angles and dihedral combinations to validate the glide polar model.

If TC-02 shows T/W < 1.0 at all conditions:

- The aircraft cannot hover in free air in the conventional sense.
- Forward flight is still possible — the aircraft needs airspeed to generate enough lift.
- The STOL and inclined stroke plane modes may still provide near-hover capability — this is tested in TC-03 with the aircraft pitched at 52 degrees nose-up.
- Do not be discouraged — many successful ornithopters do not hover and are still highly capable aircraft.

SECTION 12 — TC-03 DYNAMIC THRUST TEST — COMPLETE PROCEDURE

12.1 What TC-03 Measures

TC-03 measures how the thrust changes as the aircraft moves forward through the air. In TC-02 the aircraft was stationary. In TC-03 it moves at different speeds (typically 0 to 15 m/s in increments). This produces the CT-J curve — a graph showing how thrust coefficient (CT) varies with advance ratio (J). J is the ratio of forward speed to wing-tip speed. When J is low (slow forward speed, fast wings) efficiency is high. As J increases (faster forward flight) the thrust coefficient typically decreases.

TC-03 is usually done by mounting the aircraft on a vehicle (car or cart) and driving at different speeds while measuring force. An alternative is a wind tunnel if one is available.

12.2 TC-03 Procedure Summary

- Mount aircraft on vehicle with load cell, same as TC-02 but on a moving vehicle.
- Drive at each test speed (2, 4, 6, 8, 10, 12, 15 m/s) and hold steady for at least 10 seconds.
- The load cell records force at each speed.
- Results automatically feed into the stroke plane controller model.

```
>>> results = w.tc03_dynamic_thrust_test( ... pi_handle=pi, ... odrive_axes=axes, ...
loadcell=lc, ... pitot_fn=pitot_reader, # function returning m/s ... )
```

After TC-03, the full aerodynamic model is calibrated to this specific aircraft at its actual weight and with its actual wings. All optimizer loops now have accurate baseline data.

SECTION 13 — FIRST FREE FLIGHT — CHECKLIST AND PROCEDURE

■ Do not attempt free flight until TC-02 and TC-03 are complete and all calibrations have been verified. Do not fly in winds above 5 m/s for first flights. Always fly in a clear open area well away from people.

13.1 Pre-Flight Checklist

Complete every item before each flight. Sign off each:

■ Battery voltage	> 15.0V (fully charged = 16.8V)
■ All servo connections	Verified, not loose
■ CAN bus connections	Verified, terminators in place
■ Wing hinges	Free movement, no binding
■ Rope/capstan tension	Correct — not slack, not overtight
■ Load cell removed	Test rig disconnected for free flight
■ Pixhawk GPS lock	At least 8 satellites, HDOP < 2.0
■ Startup check passed	w.mco_startup_check() — all green
■ Ground station connected	Laptop shows live telemetry
■ Area clear	No people within 30 metres

13.2 First Flight Sequence

Flight 1 should be the minimum possible to verify basic controllability. Approximately 30-60 seconds.

- **Low hover/taxi test:** With the wave lever at approximately 50% (trim), advance slowly and observe whether the aircraft lifts off. If it does not lift off, this confirms forward flight is required and you proceed to a takeoff run.
- **First takeoff:** Advance the wave lever toward 60-70%. The aircraft should begin to lift. Allow it to climb to approximately 5 metres AGL. Do not attempt any manoeuvres.
- **Level flight check:** Find the lever position where the aircraft flies level (Ps near zero on the ground station). This confirms the trim point.
- **Landing:** Reduce lever slowly. As airspeed drops below 6.3 m/s the STOL back-side controller activates (shown on ground station). Use the lever to control descent rate. Land gently with lever at minimum.

13.3 Polar Survey (First Flight)

Once the aircraft is confirmed flyable and stable, start the autonomous polar survey on the second flight at a safe height (minimum 30 metres AGL):

```
# On laptop ground station mco> survey 35
```

The aircraft will automatically run through its test sequence — approximately 12 minutes. Do not intervene unless there is a safety issue. The survey updates all aerodynamic model constants from actual flight data.

SECTION 14 — WHAT THE OPTIMIZER IS DOING AND HOW TO READ IT

14.1 Understanding the Optimizer Display

The ground station shows optimizer status at all times. Here is what each indicator means:

Ps (Specific Excess Power)	The most important single number. Positive Ps means the aircraft has more thrust than needed to maintain level flight — it is gaining energy. Negative means it is losing energy. At trim, Ps is near zero. Unit: metres per second.
L1-L4 frozen	If any loop shows "frozen" it means a safety event has stopped the optimizer from changing anything. This is correct behaviour during violent manoeuvres. The optimizers resume automatically when safe.
Discovery ON/OFF	When ON, the aircraft is actively experimenting with new wing configurations. This is safe — the safety limits always apply even during discovery. The aircraft will not try configurations that could cause a stall or structural damage.
L6 candidates	The number of lever zone remap proposals the system has accumulated. When this is > 0, the system has found a better mode for one or more lever zones. Type "zones" in the ground station to see details and "remap ZONE" to confirm.
corr/min (Loop 7)	Lever correction frequency — how often the pilot makes a direction reversal on the lever. Lower is better. Target is below 2 corrections per minute. As Loop 7 learns, this should decrease over flights.

14.2 What Normal Looks Like

During a normal stable flight you expect to see:

- Ps between -0.1 and +0.3 during level flight
- All loops running (not frozen)
- Discovery OFF unless you have explicitly enabled it
- corr/min below 3
- CL_MAX and Vs values close to the TC-02 measurements

14.3 When to Intervene

Only intervene in the optimizer if you see:

- Ps consistently very negative (below -1.0) during level flight — indicates something is wrong with the calibration or the aircraft is too heavy.
- L6 proposing to retire a mode you use regularly — inspect the proposal before confirming.
- Discovery mode finding highly unusual configurations — inspect before locking them in.

In all other cases, let the optimizer run. It is improving the aircraft with each flight.

SECTION 15 — EMERGENCY PROCEDURES

■ These procedures must be memorised before first flight. In an emergency there is no time to look them up.

15.1 Loss of Control

- Move wave lever to minimum immediately.
- Level the wings with the roll stick.
- Allow the aircraft to descend and land.
- Do not attempt to recover from unusual attitudes below 15 metres AGL.

15.2 Runaway Aircraft (radio link lost)

The RC watchdog will automatically activate if radio signal is lost for more than 1.5 seconds. The aircraft will enter Return to Home mode and fly back to the launch point at 15 metres altitude, then descend and land.

If the RTH appears to be malfunctioning:

- The aircraft will land automatically after RTH_LAND_TIMEOUT seconds (default 30).
- Stand clear of the landing area.

15.3 Fire / Battery Damage

- Do not touch a burning LiPo battery.
- Use a Class D fire extinguisher (dry sand also works).
- Do not use water on a LiPo fire.
- Evacuate the area — LiPo fires produce toxic fumes.

15.4 Abort TC-02 or TC-03 Test

To immediately stop a running test from the laptop:

```
mco> abort
```

Or from the Pi terminal: Ctrl+C in the test script.

To cut power immediately: disconnect the battery.

SECTION 16 — GLOSSARY — EVERY TECHNICAL TERM EXPLAINED

AoA (Angle of Attack)	The angle between the wing chord (a line from leading edge to trailing edge) and the direction of airflow. Higher AoA = more lift, up to a limit (stall).
Advance Ratio (J)	Forward speed divided by wing-tip speed. J=0 means hovering. Higher J = faster forward flight.
Baud rate	Speed of serial communication, measured in bits per second. 921600 baud = 921,600 bits per second.
BEC (Battery Eliminator Circuit)	A voltage regulator that converts the main battery voltage (14.8V) to a lower regulated voltage (5V or 7.4V) for servos and electronics.
Boyd Cruise	The most efficient forward flight mode. Named after John Boyd (OODA loop creator) whose energy maneuver theory underlies the flight software's energy management.
CAN-FD	Controller Area Network Flexible Data rate. A high-speed two-wire serial bus used to communicate with the ODrives. Robust against electrical noise.
Cascade Phase	The timing offset between consecutive wing pairs. Optimal phase allows each pair to recover energy from the wake of the pair ahead.
CF1 (Clap-and-Fling)	A wing motion technique where the wings come very close together at the top or bottom of each stroke, then fling apart. Produces extra lift through vortex enhancement. Used at low speeds for maximum thrust.
CT (Thrust Coefficient)	A dimensionless number that characterises thrust efficiency. Higher CT = more efficient thrust production.
Dihedral	The upward angle of the wings relative to horizontal. Positive dihedral (tips higher than roots) gives stability. Negative dihedral (anhedral) aids maneuverability.
GEF (Ground Effect Flight)	When an aircraft flies within one wingspan of the ground, the downwash is disrupted and lift increases. The MCO uses this zone deliberately for efficient low-altitude operations.
GPIO	General Purpose Input Output. The 40 programmable pins on the Raspberry Pi used to control servos and read sensors.
HX711	A small amplifier chip that reads load cell signals and converts them to digital values the Raspberry Pi can read.
IFR (Instrument Flight Rules)	Rules governing flight in poor visibility using instruments rather than visual references. The MCO is designed to be capable of precise altitude and speed control to IFR standards.

IMU (Inertial Measurement Unit)	Sensors measuring acceleration and rotation. The Pixhawk's IMU tells the flight computer which way the aircraft is pointing and how fast it is rotating.
Lever / Wave Lever	The pilot's primary thrust control. Analogous to the throttle in a conventional aircraft. Controls energy state.
LiPo	Lithium Polymer battery. High energy density. Used in this aircraft for its light weight and high current output. Requires careful handling.
MAVLink	A lightweight communication protocol used between the Pixhawk and the Raspberry Pi to transmit sensor data and commands.
Metachronal Cascade	The timing pattern where each wing pair beats slightly after the previous one. Creates a wave-like motion through the wing pairs. Inspired by dragonfly flight.
ODrive	A high-performance motor controller. Takes position commands from the Pi and drives the flap motors with precise current control.
Pitot Tube	A forward-facing tube that measures airspeed by sensing the difference between total pressure (from the airflow) and static pressure.
Proverse Yaw	When the aircraft turns in the same direction as it banks, unlike the adverse yaw of conventional aircraft. Flapping wings naturally produce proverse yaw, which aids coordinated turns.
Ps (Specific Excess Power)	A measure of whether the aircraft has more power than needed for level flight. Positive Ps = excess power available to climb or accelerate. Negative = deficit.
PWM (Pulse Width Modulation)	The signal format used to control servos. A pulse is sent repeatedly; the width of the pulse (in microseconds) determines the servo position.
Sector Wheel	The disc-shaped mechanical component at each wing root that converts motor rotation into wing flapping. Has 200 degrees of total travel (±100 degrees from centre).
Split Stroke	A configuration where port wings use the bottom half of the flapping arc and starboard wings use the top half. Used for knife-edge flight and banked altitude control.
Stall	The condition where the wing angle of attack exceeds the critical angle and lift suddenly decreases. The MCO stall speed is approximately 4.85 m/s.
STOL (Short Takeoff and Landing)	Flight technique using maximum lift at minimum speed for precision takeoff and landing in short distances.
Stroke Plane	The imaginary flat surface through which the wing tips travel during flapping. Tilting the stroke plane redirects the thrust vector.

T/W (Thrust-to-Weight ratio)	Total thrust divided by aircraft weight. T/W > 1.0 means the aircraft can hover. T/W < 1.0 means it needs forward speed to fly.
Vne (Never Exceed Speed)	The maximum speed the aircraft must never exceed for structural safety reasons. For the MCO-1D: 28 m/s.
Vs (Stall Speed)	The minimum speed at which the aircraft can maintain level flight. Below this speed, conventional lift is insufficient to support the aircraft weight. MCO-1D Vs: approximately 4.85 m/s.
WFAOA (World-Frame Angle of Attack Correction)	A system that adjusts pitch servo commands based on bank angle so the wings always generate upward force regardless of how the aircraft is banked.

End of MCO-1D Commissioning Manual

For questions, corrections, or updates to this document, refer to the programme development log.

SECTION 17 — GUST RESPONSE SYSTEM

17.1 Overview

The Wave Library includes a complete eight-layer gust response system that activates automatically whenever turbulence or atmospheric disturbances are detected. The pilot does not need to do anything — the system runs continuously in the background and the aircraft responds to gusts faster than any human pilot could react.

The system operates on four timescales simultaneously, mirroring how dragonflies and birds respond to wind disturbances:

- **Under 10ms (passive):** Wing compliance mode allows wings to deflect slightly beyond their commanded position when a gust hits, absorbing the impulse mechanically before any electronics can respond. This is the equivalent of a bird wing acting as a suspension system.
- **10ms (IMU fast path):** A dedicated thread runs at 100Hz (independent of the main 50Hz control loop) and detects roll acceleration spikes. When a sideways gust tries to roll the aircraft, the split stroke fraction is adjusted within the next wingbeat — before the full tick cycle has even run.
- **20ms (tick-level):** The main control loop applies GR-1 through GR-7 modes — amplitude boosting on downdrafts, amplitude reduction on updrafts, per-wing sweep and dihedral adjustment, cascade phase widening in sustained turbulence, and quadrant independence for diagonal 3D gusts.
- **Seconds to minutes (Loop 8):** The gust environment characteriser tracks turbulence intensity history, detects recurring patterns (thermals, ridge crossings, valley channelling), and pre-positions wing geometry before the next predicted gust arrives.

17.2 The Eight GR Modes

GR-1 Amplitude Shield	Detects downdraft (accel_z drops below 0.8g). All eight wings increase amplitude by up to 20% to generate more upward force. Decays automatically.
GR-2 Roll Asymmetry Shield	Detects sideways roll gust from roll rate sensor. High-side wings increase amplitude and deploy sweep. Low-side wings reduce amplitude. Generates restoring roll moment within one wingbeat. All DOF used.
GR-3 Longitudinal Group Sync	In strong turbulence (intensity > 0.5): front pairs (W1+W2) synchronise their cascade phase, rear pairs (W3+W4) do the same. Maximum simultaneous force.
GR-4 Active Dihedral Boost	In sustained turbulence: all wings gradually increase positive dihedral angle, improving passive roll stability. Decays smoothly when air calms.
GR-5 Wing Tuck Equivalent	Detects strong updraft (accel_z > 1.3g). Reduces amplitude on all wings to shed lift, preventing the aircraft being thrown upward uncontrollably.

GR-6 Cascade Absorption	In sustained turbulence: widens the cascade phase spacing between pairs. Spreads peak forces across more of the stroke cycle, reducing force oscillations.
GR-7 Quadrant Independence	Detects diagonal 3D gusts (simultaneous roll + pitch accelerations). Front-port and rear-starboard wings increase amplitude; opposite quadrant decreases. Three-dimensional torque generation.
GR-8 Pre-Adaptive Morphing	Pattern recognition: if the same gust sequence has occurred 3+ times (e.g. thermals every 8 seconds), the system pre-positions wing geometry 2.5 seconds before the next predicted event.

17.3 Loop 8 — Gust Environment Characteriser

Loop 8 runs every tick and maintains a 90-second history of gust events. It computes a turbulence intensity score (0 to 1) and watches for patterns in the timing and magnitude of events. As the aircraft flies more hours in a given terrain, it learns that terrain aerological signature and begins anticipating gusts before they arrive.

The ground station shows: turb_intensity, pattern (if detected), and next_event_s (seconds until predicted next gust). These are in the GR panel of the telemetry display.

17.4 IMU Fast Path — Starting It

The IMU fast path thread is started by calling its start function with an IMU reader callback. Add this to your startup script after gs_start():

```
>>> import mco_wave_library as w >>> # Define a function that reads IMU data >>> #
(replace with your actual MAVLink or IMU read) >>> def read_imu(): ... # Read from Pixhawk
via MAVLink ... msg = mavlink_conn.recv_match(type="ATTITUDE") ... if msg: ... return
{"roll_dps": math.degrees(msg.rollspeed), ... "accel_z": accel_z_from_imu()} ... return
{"roll_dps": 0.0, "accel_z": 1.0} >>> w.imu_fast_path_start(read_imu)
```

■ The fast path runs at 100Hz by default. To upgrade to 400Hz connect the Pixhawk IMU data-ready pin to a Pi GPIO and replace time.sleep() with GPIO.wait_for_edge() in the thread.

SECTION 18 — NEW FLIGHT MODES

18.1 Backward Flight Mode

The MCO-1D can fly slowly backward by reversing the cascade phase direction and inverting the wing angle of attack. This is useful for SAR operations where the aircraft needs to back away from an obstacle without turning around in a confined space.

How to enter	While hovering or flying very slowly (below 4 m/s), push full aft stick and hold. The aircraft detects this as a backward flight request and automatically inverts the wing kinematics.
Speed limit	Maximum 4 m/s backward. Above this the S1223 airfoil inverted performance is insufficient to maintain altitude. The mode exits automatically if backward speed exceeds limit.
Performance	The S1223 airfoil produces approximately 31% of its normal lift when inverted. The library compensates with increased frequency and reduced amplitude, but backward flight is power-intensive. Expect battery consumption 2-3x normal cruise.
Bank limit	Backward flight is only available within 25° of wings-level. Attempting backward flight at steep bank angles is blocked.
To exit	Release aft stick or advance lever. The aircraft returns to normal forward flight configuration automatically.

18.2 Inclined Hover (Pending TC-02 Confirmation)

■ Inclined hover is not available until TC-02 body pitch test at 52° is completed and inclined_hover_calibrate() is called with the results. The mode is blocked by software until feasibility is confirmed.

Inclined hover means the aircraft pitches its nose up 52° and flaps its wings in a tilted stroke plane. The upward component of the wing force supports the aircraft weight; the forward component provides slow forward motion. This gives a helicopter-like loiter posture where the camera looks forward and slightly down.

After TC-02, call this once to unlock the mode:

```
>>> w.inclined_hover_calibrate( ... tc02_tw_at_52deg=, ... tc02_freq_hz= ... )
```

Required T/W in the stroke plane: 1.623 (= 1/cos(52°)). If TC-02 shows T/W above this, inclined hover is available. If below, the mode reports the minimum forward speed needed.

18.3 Inverted Recovery (Automatic)

If the aircraft rolls past 120° of bank (going inverted), the inverted recovery system activates automatically without any pilot input. The pilot does not need to do anything. The system:

- Phase 1 (0 to 0.1 seconds): confirms attitude, commands maximum frequency, determines which direction to roll back toward wings-level.
- Phase 2 (0.1 to ~1.5 seconds): maximum amplitude asymmetry between port and starboard wings drives a fast roll back toward wings-level. Typically 4-6 wingbeats.
- Phase 3 (bank below 90°): hands off to the high-G pitch-up primitive to recover altitude lost during the inversion.

■ Inverted recovery requires minimum 15m AGL to complete safely. Do not fly inverted manoeuvres below 30m AGL. The system will ABORT if height is insufficient and the aircraft may not recover before ground contact.

SECTION 19 — SOFTWARE UPDATES AND PERFORMANCE

19.1 Current Library Statistics

Library size	27,090 lines, 432 functions, 1.05 MB source
Import time	~323ms on the Pi (once at startup only)
Tick time (Pi 4)	~2ms per tick (10% of 20ms budget at 50Hz)
RAM usage	~2.3MB peak (0.056% of Pi 4 with 4GB RAM)
Storage	1.8MB total on microSD (0.006% of 32GB card)
Threads	5-6 daemon threads, all IO-bound

19.2 Mode Hints System

A new Mode Hints system provides all seven degrees of freedom (sweep, dihedral, AoA, amplitude, cascade phase, split stroke bias, and frequency) as starting-point recommendations for each flight mode. The optimizer uses these as warm starts, reducing convergence time by approximately 50% compared to starting from the nominal defaults each time.

The pilot does not interact with mode hints directly. They operate automatically. On the ground station display, the mode_hints panel shows which mode is currently active and what hint values are being applied.

19.3 Updated Ground Station Commands

gr_status	Show gust response mode states and turbulence intensity history
l8_status	Show Loop 8 pattern library and current predictions
split_status	Show split stroke fractions per wing and compliance gain
imu_fast_path_status	Show 100Hz IMU thread status and trigger count
inclined_hover_calibrate	Unlock inclined hover after TC-02 body pitch test
aero_cal_status	Show aerodynamic calibration state (unchanged from v1)
lm6_zone_report	Show lever zone map and pending remap candidates (unchanged)

19.4 Updated Pre-Flight Checklist (additions to Section 13)

■ **IMU fast path started**	Call w.imu_fast_path_start(read_imu) in startup script. Verify: imu_fast_path_status shows Running=True.
■ **Gust response armed**	Ground station shows turb_intensity=0.0 in calm conditions. GR modes will activate automatically when needed.

■ **Mode hints active**	Ground station mode_hints panel shows current intent and sweep/phase hints. Confirms system is reading lever correctly.
■ **Inclined hover (if applicable)**	Only after TC-02: confirm inclined_hover_feasible() returns True. If None: TC-02 not yet run. If False: forward speed required.

19.5 TC-02 Update — Body Pitch Test

The TC-02 procedure now includes a mandatory body pitch sweep at 0°, 15°, 30°, 45°, and 52° nose-up. This directly measures inclined hover feasibility. After the test completes:

```
>>> results = w.tc02_body_pitch_test(pi, axes, lc, tilt_rig_fn) >>> # Find the 52° result
>>> tw_52 = results["results"][(52, 3.0)]["TW"] # at 3Hz >>> freq_52 = 3.0 # frequency
used in test >>> w.inclined_hover_calibrate(tw_52, freq_52)
```

■ If the tilt rig function is None, the test pauses and asks you to manually tilt the rig to each angle and press Enter. A simple tilting platform (plywood wedge at known angles) is sufficient for initial characterisation.

SECTION 20 — HIGH-SPEED SPRINT AND PROGRESSIVE WING STOWING

20.1 The Sprint Speed Problem

A SAR vehicle that cannot reach its destination quickly is less useful than one that can. The MCO-1D has been designed from the outset to combine the slow-flight precision needed for on-site operations with the transit speed needed to reach remote locations quickly. This section describes how the sprint system works and what speeds are achievable.

20.2 High-Speed FSF — Dynamic Sweep Per Stroke

The High-Speed Flap-Sweep-Forward (HSF) system is the key to higher sprint speeds. Rather than holding the wings at a fixed sweep angle during sprint, the sweep servo moves within each individual stroke:

- **Downstroke:** Wings sweep forward (toward 25-55° depending on speed) so the leading edge meets the combined airstream at the optimal angle. This generates a direct forward thrust component rather than relying entirely on stroke-plane tilt.
- **Upstroke:** Wings sweep aft (toward 75-85°) to reduce the drag of the recovery stroke, just as a bird folds its wing on the upstroke.

The sweep angle changes within each stroke cycle are invisible to the pilot. The aircraft simply feels faster and more responsive at the sprint lever position. Thrust improves by an estimated 15-35% over fixed-sweep sprint configurations.

■ The previous sprint_params() function used sweep=90° (wings pointing sideways). This has been replaced by the HSF system. The old 90° configuration is no longer used.

20.3 Progressive Wing Stowing

As airspeed increases, the rear wing pairs are progressively folded aft to reduce drag. The stowed pairs are swept to approximately 100° (past perpendicular) where they are aerodynamically unloaded. Only the front pairs continue to generate lift and thrust.

Below 15 m/s	All 4 pairs active. Normal flight configuration. Stall speed: 5.2 m/s.
Above 15 m/s	W4 pair folds. 3 pairs active (W1, W2, W3). Stall speed rises to 6.4 m/s. Drag reduces significantly.
Above 25 m/s	W3 pair also folds. 2 pairs active (W1 and W2). Stall speed rises to 9.7 m/s. Must not slow below 12 m/s in this configuration.
Above 45 m/s (sprint only)	W2 pair folds. W1 pair only. Stall speed approximately 14 m/s. Only in dedicated sprint mode — not for general use.

■ When wing pairs are stowed the stall speed rises significantly. The system automatically restores all pairs if airspeed drops within 1.25x of the elevated stall speed. The pilot will see a STOW SAFETY alert on the ground station if this happens.

20.4 Wing Pair Role Differentiation

When pairs are active during sprint, each pair is assigned a specific aerodynamic role rather than all doing the same job:

W1 — Front pair (longest)	LIFT pair. High angle of attack, forward sweep, maximum span. Keeps the aircraft flying at the elevated stall speed.
W2 — Second pair	THRUST pair. Moderate angle of attack, dynamic HSF sweep optimised for forward force generation. Provides the majority of forward thrust.
W3 — Third pair	Secondary thrust pair. Similar to W2 but lighter loading. Folds at 25 m/s.
W4 — Rear pair (shortest)	Folds at 15 m/s. Contributes primarily at low speed where all lift area is needed.

This mirrors the dragonfly forewing/hindwing role split, where forewings specialise in lift and hindwings in thrust during fast forward flight.

20.5 Sprint Speed Estimates

All 4 pairs, HSF dynamic sweep	35-50 m/s (126-180 km/h)
3 pairs active (W4 stowed), HSF	40-55 m/s (144-198 km/h)
2 pairs active (W3+W4 stowed), HSF	50-70 m/s (180-252 km/h)
1 pair active (W1 only)	70-100+ m/s (record attempt only)
Structural ceiling (Vne)	28 m/s current — may be raised after TC testing

■ All sprint speeds above the current Vne of 28 m/s require structural validation of the rope drive, sector wheels, and wing fairings at those dynamic pressures. The Vne will be raised incrementally as testing confirms structural integrity. TC-03 will measure the CT-J curve and define where thrust actually falls off with increasing airspeed.

SECTION 21 — AUTONOMOUS MODE DISCOVERY AND LEVER EVOLUTION

21.1 What This System Does

The lever map is not fixed. The aircraft continuously experiments with new wing configurations during normal flight and permanently inserts the best discoveries into the lever map, replacing less effective configurations. Over time the lever map evolves from the library design assumptions toward empirically proven optima for this specific aircraft.

The pilot experiences none of this. The lever still moves between 0% and 100%. Every position still responds smoothly and predictably. The aircraft simply improves with every flight, with no intervention required.

21.2 How Discovery Works

- **Search space (32 dimensions):** The discovery system explores a space that includes global parameters (frequency, cascade phase, global sweep, AoA, amplitude, dihedral) AND per-pair parameters for all four wing pairs independently (per-pair AoA offset, sweep offset, amplitude scale, phase offset, split stroke bias) AND HSF sweep profile shape.
- **Dwell and evaluate:** Each configuration is held for 8 seconds while the system measures mean Ps. If the result beats the previous best at that lever position by more than 8%, the configuration is registered as a discovered mode with a generated name (DISC_0001, DISC_0042, etc.).
- **Promote to lever map:** The discovered mode is immediately inserted into the live lever map at its lever position. All future calls to that lever position return the discovered mode instead of the library default.
- **Continuous competition:** Better configurations continue to displace weaker ones. The lever map never stops improving.
- **Smooth transitions:** All transitions to and from discovered modes use the same bell-curve smooth_step system as library modes. The pilot feels no discontinuity.

21.3 Enabling and Monitoring

Autonomous discovery is controlled from the ground station:

```
auto_disc_enable / auto_disc_disable / auto_disc_status / disc_registry / lever_map_live
```

The auto_disc_status output shows:

Configs tried	Total configurations evaluated since enable
Modes promoted	How many have been inserted into the live lever map
Live map size	Number of lever positions with a discovered mode outperforming the library default
Registry size	Total named discovered modes (including superseded ones)

■ Autonomous discovery requires height above DISCOVERY_MIN_H_M (default 15m AGL) and is automatically suspended during any violent manoeuvre, gust event above threshold, or when stall risk is detected. It resumes automatically when safe conditions return.

SECTION 22 — LOOP 9: META-OPTIMIZER

22.1 What Loop 9 Does

Loops 1-8 are tactical optimizers — each tunes its own assigned parameters within a fixed boundary. Loop 9 is the strategic optimizer. It observes the outputs of all other loops and adjusts the parameters that define those boundaries. Where Loops 1-8 optimise the aircraft, Loop 9 optimises the optimizers themselves.

22.2 Three Objectives in Priority Order

- **1. Safety (highest priority):** Stability margins, stall clearance, gust response adequacy. If a safety event occurs, all performance-category parameters are frozen for 5 minutes. Safety parameters are never frozen — they may need to increase in response to the same event.
- **2. Performance:** Speed, climb rate, sprint capability. Loop 9 learns the actual crossover speeds at which wing stowing becomes beneficial, the actual optimal AoA for HSF at each speed, and raises the frequency ceiling as the hardware proves itself. Performance parameters can only be adjusted when no safety freeze is active.
- **3. Efficiency:** L/D, endurance, power consumption. This is the lowest priority but informs how the aircraft behaves in the cruise and loiter zones where most SAR mission time is spent.

22.3 What Loop 9 Can and Cannot Change

CAN change (with evidence)	Progressive stow crossover speeds, HSF optimal AoA, frequency ceiling (only ever upward), split stroke gain, gust response authority levels, pilot feel parameters.
CANNOT change	Stall speed, sector wheel travel limits, maximum amplitude, stall recovery constants, inverted recovery minimum height, ground effect height, any constant containing LIMIT, MAX, MIN, SAFETY, EMERGENCY, or ABORT.
Maximum change per evaluation	8% of current value. Small, evidence-based adjustments only. Each parameter needs 20 observations before any adjustment.
Evaluation interval	Every 2 minutes of flight. Slow enough that changes are not noticeable to the pilot within a flight session.

22.4 Ground Station Commands

l9_status	Print all parameters Loop 9 can tune, their current values, observation counts, and hard limits.
l9_freeze PARAM	Prevent Loop 9 from changing a specific parameter. Example: l9_freeze STOW_THRESHOLD_W3_MS
l9_unfreeze PARAM	Allow Loop 9 to resume tuning a frozen parameter.

l9_reset PARAM	Reset a parameter to its factory default value and clear all observations for that parameter.
l9_disable	Disable Loop 9 entirely for this session.
l9_enable	Re-enable Loop 9 after disable.

22.5 Updated Software Statistics (Revision 3)

Library size	29,126 lines (1.12 MB source)
Optimizer loops	9 loops (Loop 9 = meta-optimizer added)
Discovery dimensions	32 (was 14 in previous version)
Live lever map	Grows autonomously — up to 200 positions
Tick time (Pi 4)	~2.2ms per tick (11% of 20ms budget)
RAM usage	~2.5MB peak (unchanged from v2)

22.6 Updated Pre-Flight Checklist Additions

■ **Loop 9 status**	Run: l9_status on ground station. Verify no unexpected parameters outside expected ranges. Note any recent adjustments in the log.
■ **Autonomous discovery intent**	Decide before flight whether to enable autonomous discovery. Enable with: auto_disc_enable Disable if you want repeatable baseline performance: auto_disc_disable
■ **Live lever map**	Check lever_map_live to see if any discovered modes are active. If you see unexpected DISC_NNNN entries at positions you will use, review them with disc_registry.
■ **Progressive stow awareness**	If flying above 15 m/s: the aircraft will automatically stow W4. Brief the pilot that stall speed is elevated above 15 m/s and the aircraft must not slow below 8 m/s in that configuration.

SECTION 23 — GROUND OPERATIONS — COMPLETE SYSTEM

23.1 Overview

The MCO-1D has a complete ground operations system covering five distinct post-flight phases and a full holonomic ground control capability. The aircraft has castoring wheels and no powered wheels — all ground propulsion comes from the wings. This section covers every ground mode from powered taxi through storage.

23.2 Wing Configuration — Upper Hemisphere Taxi

The fundamental insight of the MCO-1D taxi system is that the wings need only stroke in the upper hemisphere — from horizontal upward — to generate both forward thrust and ground clearance simultaneously. The downstroke never crosses below horizontal, so there is no risk of ground contact regardless of amplitude.

At 30 degrees of dihedral the stroke plane is tilted. Every point in the stroke arc is well above the ground. The forward component of force (sin 30 degrees = 0.5) propels the aircraft. The upward component unloads the castoring wheels, reducing rolling resistance and preventing porpoising.

Sweep	90 degrees (wings lateral, perpendicular to fuselage)
Dihedral	30 degrees (stroke plane tilted, forward thrust component)
Sector	[0 degrees, +amplitude] — upper hemisphere only
AoA	+18 degrees forward, -18 degrees reverse
W4 clearance	556mm above ground at 30 degrees dihedral (sector_min=0)

23.3 Lever Convention on the Ground

The lever works like a conventional fixed-wing throttle while in taxi mode — the same feel pilots already know:

0 to 5%	Idle — wings stopped, aircraft holds position
5 to 10%	Barely cracked — slow crawl. Advance briefly to get rolling, ease back to hold speed.
10 to 15%	Walking pace taxi
15 to 22%	Brisk taxi
22 to 30%	Sprint taxi
Above 30%	Pre-takeoff — system arms for liftoff

■ On the ground the lever maps to the conventional taxi convention: barely crack it to get rolling, feather back to hold speed. This is identical to the fixed-wing feel described in every primary training syllabus.

23.4 Lateral Translation (Roll Stick)

The roll stick produces direct sideways translation — a capability conventional fixed-wing aircraft do not have. Port wings increase dihedral and stroke to push the aircraft right. Starboard wings stop. The result is 10.9 N of lateral force giving 3.16 m/s squared lateral acceleration.

Speed is capped at 1.4 m/s (walking speed) in both lateral directions for safety near ground crew. The amplitude scales down automatically as this speed is approached.

23.5 Yaw (Pedals) — Same as Air

The yaw pedals produce differential amplitude between port and starboard wings, giving the same feel as in-flight yaw. No relearning required. At full pedal deflection (above 85 percent) the system enters spot turn mode — starboard sectors reverse to generate backward thrust while port wings push forward. The aircraft pivots in its own length in approximately six seconds for a full 360 degrees.

23.6 Backward Movement

Backward taxi is commanded by pulling the pitch stick full back while the lever is in the taxi range (0 to 30 percent). This reverses the AoA to -18 degrees, making the upstroke push the aircraft backward. Backward movement requires at least walk-level power (lever above 10 percent) to overcome rolling resistance. Speed is capped at 1.4 m/s.

■ All backward movement, lateral translation, and combinations use the same bell-curve smoothing as flight — inputs are velocity-proportional. A slow deliberate stick movement produces a slow gradual response. A decisive input produces a faster response. Nothing jumps or lurches.

23.7 Post-Flight Sequence

1. Land	Touchdown detected automatically. Lever to idle. Aircraft decelerates on its own.
2. Taxi (if needed)	ground_ops_set_mode("taxi") from ground station. Lever barely cracked to move to parking spot. Roll stick for sideways nudges. Yaw pedals to steer.
3. Cooldown — 8 minutes	ground_ops_set_mode("cooldown"). Wings oscillate in-phase at 45 degrees dihedral, 0.8 Hz, 30 degrees amplitude. Zero net thrust. Motors cool from 80 C to 45 C. DO NOT handle motors until complete.
4. Aft stow	ground_ops_set_mode("aft_stow"). Wings sweep to 100 degrees, dihedral to +15 degrees, amplitude to zero. Sequenced: amplitude first, then sweep, then dihedral. Wings fold posteriorly along fuselage sides.
5. Disarm	Disable ODrives after aft stow settles. No power needed to maintain stow position. Wings rest against mechanical stops.

23.8 Storm Stow (Oshkosh and High Wind Parking)

■ Wings spread at 90 degrees sweep present 0.979 m squared of lateral area. A 15 m/s lateral wind generates over 100 N of overturning force against a restoring moment of only 6.8 N metres from the wheel track. The aircraft will tip over without tie-downs in any significant wind if wings are spread.

Storm stow minimises the presented area to 0.18 m squared (82 percent reduction) by folding wings to their most compact position, then disarming servos so tie-down straps carry all wind loads without fighting servo holding torque.

Storm stow config	Sweep 100 degrees (maximum aft), dihedral +20 degrees (maximum servo limit), amplitude 0 degrees, then DISARM
Below 12 m/s (23 kts)	Normal aft stow adequate
12 to 15 m/s (23-29 kts)	Enter storm stow, monitor conditions
Above 15 m/s (29 kts)	Storm stow PLUS TIE-DOWN STRAPS required
Ground station command	ground_ops_set_mode("storm_stow")

■ HARDWARE REQUIREMENT: Wing tip structural rings must be built into each wing tip for tie-down straps. This is a design requirement, not optional. Without tie-down points the aircraft cannot be safely secured in storm conditions.

SECTION 24 — GROUND TO AIR AND AIR TO GROUND TRANSITIONS

24.1 The Boundary Problem

The transition between ground operations and flight is the most safety-critical phase of any sortie. The MCO-1D has four specific improvements over naive state switching to make these transitions smooth and correct.

24.2 Automatic Takeoff Handoff

When in taxi mode and the lever advances past 30 percent for more than 0.5 seconds, the system automatically switches from ground control to flight control. The 0.5 second confirmation prevents accidental trigger from a momentary lever overshoot during brisk taxi.

From the pilot perspective: advance the lever as for takeoff. The aircraft responds as it always has. No mode switch is needed from the cockpit.

24.3 Automatic Taxi Offer After Landing

After touchdown, the system monitors deceleration. When speed drops below 1.5 m/s and holds there for 3 seconds, the system automatically switches to taxi mode. The lever immediately starts working in the conventional ground sense without any pilot action.

The 3 second delay prevents taxi mode activating during a fast landing roll that briefly slows below threshold.

24.4 Continuous Sector Blend During Liftoff

During taxi the wings operate in upper hemisphere only (sector_min = 0 degrees). In flight the full symmetric stroke is available. Previously this was a sudden jump at the mode boundary. Now the downstroke opens continuously with altitude using the same geometry as the ground clearance system:

h = 0.00 m	Sector min = 0 degrees (upper hemisphere only)
h = 0.10 m	Sector min = -12 degrees (slight downstroke opens)
h = 0.20 m	Sector min = -21 degrees
h = 0.37 m	Sector min = -36 degrees
h = 0.65 m	Sector min = -96 degrees (full stroke available)

24.5 Smooth Dihedral and Sweep at Mode Boundary

At liftoff the taxi configuration (dihedral 30 degrees, sweep 90 degrees) transitions smoothly to cruise configuration (dihedral 0 degrees, sweep 60 degrees) over approximately 2 to 3 seconds. The aircraft folds

its wings from taxi posture into cruise posture during the first seconds of climb. Nothing jumps.

24.6 Twelve State Ground Machine

The unified state machine determines which physical interpretation applies to every sensor reading and lever position. Intent (what mode the pilot commanded) is combined with physics (height, speed, acceleration) to produce an unambiguous state:

disarmed	ODrives and servos off
storm_stow	Storm parking configuration
aft_stow	Post-flight storage
cooldown	8 minute motor cooling
taxi	Ground propulsion active
landing_roll	Post-touchdown deceleration, V > 2 m/s
takeoff_roll	Ground acceleration for conventional takeoff
gef	Airborne, h = 0.04 to 0.37 m
takeoff_climb	h rising, Ps > 0, dV/dt > 0.4 m/s squared
approach	Descending, low lever, h < 30 m
flare	h < 0.5 m, decelerating
cruise	Normal flight above all restriction zones

SECTION 25 — TOUCH AND GO OPERATIONS

25.1 What a Touch and Go Is

A touch and go is a maneuver in which the aircraft lands on the runway and takes off again without coming to a full stop. After the wheels are down, the pilot advances the lever to full power while still rolling, and the aircraft lifts off again. It is used extensively in training to practise approach and landing technique efficiently.

25.2 MCO-1D Touch and Go — Key Difference

The MCO-1D touches down at approximately 5.5 m/s (1.3 times stall speed in ground effect). Rotation speed is 5.3 m/s. The aircraft is already above rotation speed when it touches down. There is no need to build speed during the ground roll — the aircraft can lift off almost immediately after the wheels touch. The runway roll exists only to allow configuration reset.

25.3 Procedure

The sequence mirrors conventional fixed-wing technique exactly. Priority order: power, then attitude, then configuration — the same as a go-around.

Approach	Normal approach in GEF zone. Lever at approach power (15 to 35 percent). Sector progressively restricted by altitude.
Touchdown	Wheels contact runway. System detects touchdown via height and accelerometer. Wings continue running — do NOT stop. Sector held in upper hemisphere to prevent bounce (equivalent to partial flap retraction).
Reconfigure (0 to 1 second)	Advance lever from approach band toward sprint. This IS the configuration change — lever combines throttle and flap retraction into one movement.
Full power (1 to 2 seconds)	Lever to 80 to 100 percent. Wings at maximum amplitude upper hemisphere. Applied over 1 to 2 seconds — smooth and deliberate, not slammed.
Liftoff	Aircraft lifts — transition_tick detects height rising. Sector_min opens progressively with altitude. Full flight authority at h = 0.65 m.
Climb out	Normal flight. Dihedral and sweep transition smoothly to cruise configuration over 2 to 3 seconds.

■ The lever IS the combined throttle-plus-flap control. Advancing from approach band to sprint band in one smooth motion is equivalent to retracting flaps to takeoff setting and advancing throttle simultaneously — as a single fluid operation. Practise this motion on the ground first.

■ Never force the aircraft back onto the runway after liftoff. If the aircraft lifts off and the pilot cuts power, the emergency lever cut response activates: stall protection holds minimum

frequency if below 10 m AGL.

SECTION 26 — EMERGENCY LEVER RESPONSE

26.1 Overview

The MCO-1D detects violent lever movements and responds with appropriate urgency — faster than the normal bell-curve smoothing but never instantaneously. Two types of violent input are recognised:

26.2 Violent Push — Emergency Thrust

Detected when the lever advances faster than 380 percent per second — covering the full range in 0.25 seconds. This is unmistakably an emergency input (obstacle, wind shear, go-around decision).

Detection threshold	0.04 normalised per tick (50 Hz)
Response time	Full amplitude in approximately 0.3 seconds
Ground response	Maximum amplitude immediately, upper hemisphere, taxi dihedral held
Air response	CF1-class thrust burst at urgency=1.0, three times normal smooth_step rate
Duration	Emergency authority for 2 seconds, then normal lever tracking resumes

26.3 Violent Cut — Emergency Abort

Detected when the lever drops faster than -380 percent per second. Three different responses depending on situation:

Ground abort	Amplitude drives to zero immediately at maximum rate. Wings stop. Equivalent to aborting a takeoff roll. Use brakes and directional control to stop.
Air — low altitude (below 10 m)	STALL PROTECTED. Wings do NOT stop. Frequency held at a minimum of 1.5 Hz regardless of lever position. This is non-negotiable — the aircraft will stall and fall if wings stop below 10 m AGL.
Air — high altitude (above 10 m)	Minimum power glide. Amplitude reduces to 20 degrees. Sweep deploys to 45 degrees for maximum glide ratio. Aircraft configures for best glide immediately.

■ A violent lever cut below 10 m AGL will NOT stop the wings. The stall protection system overrides the cut command. This is intentional and correct. Do not attempt to stop the wings at low altitude in flight — let the system configure for minimum power glide instead.

26.4 Normal vs Violent — The Distinction

A normal full-power advance takes 1 to 2 seconds (approximately 50 percent per second). A violent push covers the same range in 0.25 seconds or less. The system distinguishes these correctly. A brisk but deliberate advance is not flagged as violent — only a genuine emergency input triggers the emergency path.

■ The bell-curve smoother remains active even in emergency mode — at urgency=1.0 the rate triples but the smooth profile is preserved. There is no discontinuous jump in any parameter under any condition.

26.5 Updated Software Statistics (Revision 4)

Library size	32,584 lines (1.25 MB source)
Tick stack depth	28 layers (ground control added)
Ground states	12 unified states with hysteresis
Touch and go	Automatic phase sequencing
Emergency response	Violent push and cut detection at 50 Hz
Tick time (Pi 4)	~2.5 ms per tick (12.5 percent of 20 ms)

SECTION 27 — UNIFIED LEVER CONVENTION

27.1 One Convention, Always

The wave function lever works the same way in every situation — on the ground, in the air, during approach, during takeoff, during a touch and go. Forward is always more power. Aft is always less power, then braking, then reverse on the ground. The pilot learns one convention and it never changes.

27.2 The Lever Map

100% (full forward)	Maximum thrust — sprint, CF1, takeoff, sprint taxi on ground. Same position, same intent, in every domain.
72–100%	SPRINT zone in air. Pre-takeoff and sprint taxi on ground. Full forward is full power everywhere.
57–72%	POWERED CRUISE. Jog taxi on ground.
42–57%	STD CRUISE / TRIM. Walk to crawl taxi on ground.
50% (centre)	Trim in air — level flight hold. Stopped on ground — idle. The natural resting position.
36–42%	TRIM / light descent. Barely cracked forward of centre on ground — the primary taxi band.
25–36%	CLEAN GLIDE. Below idle on ground — slight rearward.
13–25%	APPROACH power. Backward walk on ground.
0–13%	STEEP DESCENT to MAX AIRBRAKE. Backward sprint on ground (capped at 1.4 m/s).
0% (full aft)	Maximum aerodynamic braking in air. Maximum backward taxi on ground. Never reverses in air — full aft is only brake.

27.3 The Touch and Go — One Continuous Motion

The unified lever makes the touch and go completely natural. The pilot never changes convention mid-manoeuvre:

Approach	Lever at ~20% — approach power, pulled back. Descending with control.
Flare	Pull to ~45% — reducing power, slowing descent.
Touchdown	Lever at ~50% — idle. Aircraft on ground, nearly stopped. Sector held upper hemisphere.
Go again	Push forward to 95–100% — full sprint power. Aircraft already above Vr. Lifts off immediately.
Climb	Pull back to ~80% — climb power. Natural.

Cruise	Pull back to ~60% — cruise power. The same aft motion that slowed the approach now sets the cruise.

■ The MCO-1D touches down at approximately 5.5 m/s, which is already above rotation speed (5.3 m/s). The touch and go requires almost no ground roll — push the lever forward and the aircraft lifts immediately.

27.4 Takeoff

Full forward on the ground is takeoff. The pilot advances the lever to full. The aircraft accelerates. When it has sufficient speed and lift it leaves the ground. No mode switch, no button, no separate arming required. The lever is pushed forward and the aircraft flies.

27.5 Backward Taxi

Pulling the lever aft of centre (below 50%) on the ground commands backward movement. The further aft, the faster — capped at 1.4 metres per second (walking speed) for safety. The AoA reverses while maintaining the upper hemisphere sector — the same ground clearance applies in both directions.

■ Backward flight in air is not possible on the MCO-1D. The S1223 aerofoil inverted produces only 31 percent of its normal lift, giving a sink acceleration of 6.77 m/s² — the aircraft cannot maintain altitude in reverse. Full aft lever in air is maximum aerodynamic braking only. Backward taxi works correctly because the ground supports the aircraft weight.

SECTION 28 — SNAP ROLL AND GESTURE MAPPING

28.1 Snap Roll — Proportional Direct Control

The roll stick is a direct roll rate command. The aircraft rolls at a rate proportional to stick deflection and continues rolling as long as the stick is held. When the stick returns to neutral, rolling stops. This is identical to aileron behaviour in a conventional aircraft — the stick IS the roll rate.

Stick velocity scales the response urgency. A fast stick throw produces a faster response onset. A slow deliberate roll input produces a smooth gradual response. The bell-curve smoother remains active at all urgency levels — there is no discontinuous jump under any condition.

Full left stick	Port wings 96°, starboard 0° — maximum left roll rate
Half left stick	Proportional differential — half roll rate
Neutral stick	Equal amplitude both sides — wings level, no roll
Half right stick	Proportional differential — half roll rate right
Full right stick	Starboard 96°, port 0° — maximum right roll rate

28.2 Confirmed Gesture Map

zoom_climb	Pitch stick back fast + lever at sprint (above 80%). Trades airspeed for altitude on the Ps=0 boundary.
stoop_auto	Pitch stick forward fast + lever in glide zone (below 30%). Initiates the stoop sequence.
power_dive	Pitch stick forward steady + lever above 55%. Controlled descent under power.
pullout	Automatic — Boyd zero-crossing detector. Fires when Ps crosses zero during stoop.
jump_takeoff	Lever above 92% AND pitch stick maximum on ground. Ballistic vertical launch.
inverted_recovery	Automatic — fires when bank exceeds 120 degrees. Three-phase recovery sequence.
cf1	Lever above 95% at slow speed. Maximum thrust burst — CF1 configuration.

28.3 Pending Gestures (post TC-02)

The following functions have parameters and logic built but no gesture trigger yet. All three require TC-02 test data or further operational definition before a safe trigger can be designed:

dart_left / dart_right	Rapid yaw-then-translate directional manoeuvre. Trigger design pending operational definition.

inclined_hover	Body pitch 52 degrees, sustained vertical climb. Requires TC-02 T/W measurement at 52 degrees before inclined_hover_calibrate() can be called. Gesture will be a deliberate triple-input combo.

SECTION 29 — TOUCHSCREEN PANEL — FOUR BUTTONS

29.1 Hardware

The Official Raspberry Pi 7-inch DSI touchscreen connects directly to the Pi DSI ribbon connector with four GPIO pins for power. No HDMI cable, no USB, no driver installation. Capacitive multi-touch. Approximately 80 USD. Install pygame with: sudo apt install python3-pygame

29.2 Four Buttons Only

Everything that can be automatic is automatic. Everything that can be gesture-driven is gesture-driven. The touchscreen handles only the four functions that genuinely require a deliberate operator action at the aircraft:

PRE-FLIGHT (hold 1.5s)	Runs all three startup check phases: Phase 1 geometry and PWM math, Phase 2 servo sweep with visual confirmation, Phase 3 ground-aware ODrive power check. On pass: arms ODrives, enables discovery, activates all optimizers. Discovery runs from this point forward without ever being turned off.
POST-FLIGHT (single press)	Runs post-flight checks (battery, structure, log). Then automatically sequences: 8-minute cooldown, aft stow (wings fold posteriorly), disarm. Cooldown progress shown as arc in status bar. Press once and walk away.
STORM STOW (single press)	If POST-FLIGHT cooldown is running: overrides the aft stow step with storm stow for this instance. If pressed standalone: direct path to storm config, then auto-disarm after 22 seconds. Always ends with tie-down warning in status bar when wind is above 15 m/s.
EMERGENCY STOP (no confirmation)	Instant wing stop. No hold required. In air below 10m: stall-protected — holds minimum frequency. In air above 10m: minimum power glide. On ground: amplitude to zero immediately.

29.3 Status Bar

The header shows ground state, airspeed, height, battery voltage (amber below 13.5V), turbulence level, Loop 9 status, and discovery status. The cooldown arc appears in the top right corner counting down during the post-flight sequence. A tie-down warning appears in red when in storm stow mode with wind above 15 m/s.

■ Everything else — GEF, discovery, backward taxi, all nine optimizer loops, ground state transitions, sector_min blend, dihedral transition — is fully automatic. The pilot does not interact with any of it.

SECTION 30 — AUTONOMOUS DISCOVERY AND OPTIMIZER OVERVIEW

30.1 Discovery Always On

Autonomous mode discovery is enabled at library startup and runs continuously during every flight above the minimum altitude (15m AGL). It does not require a button, a command, or any pilot awareness. The aircraft experiments with new wing configurations, measures their performance, and permanently improves the lever map with every flight.

30.2 How Modes Are Created and Promoted

Search space	32 dimensions: global parameters plus per-pair AoA, sweep, amplitude, phase, and split stroke bias for all four wing pairs independently.
Evaluation	Each configuration is held for 8 seconds while measuring mean specific excess power (Ps). If Ps improves on the best seen at that lever position by more than 8 percent, the configuration is registered as a discovered mode.
Naming	Discovered modes are named DISC_0001, DISC_0042, and so on. They are stored in the discovery registry with their configuration, score, and lever position.
Promotion	The mode is immediately inserted into the live lever map at its position. Future calls to that lever position return the discovered mode instead of the library default — provided it has been confirmed by at least 5 observations.
Competition	Better configurations continue to displace weaker ones. The lever map never stops improving. Every mode — library or discovered — competes on equal footing based on measured Ps.

30.3 The Nine Optimizer Loops

Loop 1	Stroke AoA — peak CL every stroke
Loop 2	Cascade phase — wake capture efficiency
Loop 3	Surge offsets — front/rear thrust balance
Loop 4	Banked configuration — arc centres and widths
Loop 5	Yaw modality weights — turning efficiency
Loop 6	Lever zone mode assignments — best mode per zone
Loop 7	Lever feel — pilot correction frequency
Loop 8	Gust characteriser — pattern recognition, GR-1 to GR-8
Loop 9	Meta-optimizer — tunes parameters governing Loops 1-8

Loop 9 is the strategic layer. It observes all other loops and adjusts the parameters that define their operating boundaries — stow crossover speeds, frequency ceiling (only ever upward), gust response authority, pilot feel constants. All adjustments are small (8 percent maximum per evaluation) and evidence-based (20 observations minimum). Safety parameters are never reduced.

30.4 Updated Software Statistics — Revision 5

Wave Library	33,472 lines (1.28 MB)
Panel application	435 lines
Optimizer loops	9 (Loop 9 = meta-optimizer)
Discovery dimensions	32 per-wing parameters
Ground states	12 unified states with hysteresis
Tick stack depth	30 layers
Tick time (Pi 4)	~2.5 ms (12.5 percent of 20 ms budget)
RAM usage	~2.8 MB peak

SECTION 31 — FOUR-LAYER EXPLORATION SYSTEM

31.1 The Gap the Optimizer Cannot Cross

The nine optimizer loops and the 32-dimension discovery system are excellent at finding the best point within a defined search space. But the shape of that search space was designed by humans. A gradient optimizer cannot discover that the search space itself has the wrong shape. It cannot invent a parameter that does not already exist in its search dimensions.

To discover genuinely novel configurations — ones that are aerodynamically valid but that no human thought to try and no animal has evolved — a different approach is needed. The four-layer exploration system addresses this directly.

31.2 The Four Layers

Layer 1 — Gradient Exploitation	The existing optimizer: Loops 1-9 plus 32-dimension autonomous discovery. Finds the best point within the current search space. Runs continuously and always. Never stops improving what it already knows.
Layer 2 — Biomimetic Hypothesis Injection	Twelve novel configuration types derived from biological research. Each introduces a new parameter dimension that does not exist in the current search space — a qualitatively different idea, not just a new value for an existing parameter. Dragonfly, bat, manta ray, pterosaur, cetacean, swift, locust.
Layer 3 — Evolutionary Search	Takes successful DISC_NNNN modes and mutates them. Changes 1-4 parameters simultaneously by a Gaussian random amount. Crossbreeds two successful modes — each parameter taken from one parent or blended between them. Finds combinations that gradient search cannot reach because they require crossing a local minimum to get there.
Layer 4 — Random Safety-Gated Search	Pure uniform random sampling of the full 32-dimensional parameter space. No biological inspiration, no parent configuration, no gradient direction. This is the layer that discovers what nature has not tried — configurations that are aerodynamically valid but that no evolutionary path has ever produced. Most trials fail. The ones that improve specific excess power are kept permanently.

31.3 Scheduling — 80 Percent Option B, 20 Percent Option C

The four layers need a scheduling policy. The MCO-1D uses a combination of two approaches:

Option B (80 percent of exploration time): Layers 2, 3, and 4 only activate when Layer 1 has converged at the current lever position — when the gradient optimizer has produced no meaningful improvement for two consecutive minutes. If Layer 1 is still finding improvements there is no point running random search alongside it. This mirrors how good researchers work: refine what you have until refinement stops paying,

then try something genuinely different.

Option C (20 percent): Curiosity-driven. The system tracks how long since each lever position produced a novel discovery. Positions that have not improved in ten or more minutes receive proportionally more Layer 3 and 4 time regardless of convergence state. Positions the optimizer has ignored get probed more aggressively over time.

31.4 Safety Framework

Every trial from every layer — including completely random Layer 4 trials — passes through the same safety gates before it can run and is continuously monitored while it runs.

Hard walls (cannot be crossed)	Frequency at or below 3.5 Hz for novel configurations. Amplitude at or below 96 degrees. Airspeed at or above 1.3 times stall speed. Height at or above 15 m AGL. Turbulence intensity below 0.15. Motor current below 85 percent of limit. Per-pair frequency differential below 1.5 Hz.
Soft abort (trial ends, result discarded)	IMU vibration spike during trial. Unexpected motor current increase. Specific excess power drops sharply below baseline. Any gust response GR-1 through GR-8 fires. Unexpected altitude loss detected.
Trial conditions	Duration: 5 seconds maximum (shorter than standard 8s). Maximum 3 trials per novel configuration before discard. Calm air only: turbulence intensity below 0.15. Moderate speed only: not sprint, not hover.

31.5 Full Autonomy

The entire system runs without permission after the pre-flight arm. Discoveries feed into crossbreeding. Successful mutations become parents for further mutations. Random successes from Layer 4 get named, crossbred in Layer 3, and gradient-refined in Layer 1. The chain is closed and perpetual. The aircraft improves every flight, indefinitely.

■ Ground station command: exploration_status — shows trial counts, promotion counts, and active layer for each of the four layers.

SECTION 32 — TWELVE BIOMIMETIC HYPOTHESES

32.1 The Principle of Biomimetic Injection

Natural organisms have evolved flight solutions over hundreds of millions of years. These solutions are physically valid and experimentally proven. However, they cannot be transferred wholesale to the MCO-1D because the structural properties, materials, and size are completely different.

The correct approach is to extract the aerodynamic PRINCIPLE from the biological solution — not the raw kinematics — and translate it into a testable MCO-1D configuration. The optimizer then validates the principle empirically in flight.

32.2 Active Hypotheses (Safe to Test Now)

H-01 — Dragonfly Phase Switching	Source: Reavis and Luttges 1988, Zheng et al. Dragonflies flap in-phase when maximum force is needed (takeoff, rapid climb) and antiphase for stability. In-phase increases peak lift from 2.3 to 6.3 times body weight. Phase shift of 45 degrees maximises thrust coefficient. MCO-1D test: vary cascade phase from 0 to 180 degrees in 10-degree steps, measuring Ps at each.
H-02 — Bat Advance Rotation	Source: PLOS ONE 2019, Biology Open 2012. Bat wings begin pitching toward the next stroke AoA before the current stroke ends — advance rotation. This increases lift by 2.5 times versus late rotation. MCO-1D test: make AoA a function of stroke phase rather than a fixed value. New parameter: AoA advance angle.
H-03 — Manta Spanwise Wave	Source: MDPI 2022 — manta achieves 89 percent propulsive efficiency. Pectoral fin shows both spanwise and chordwise wave motion. Phase gradient from root to tip. MCO-1D test: within-pair phase gradient using existing per-wing phase offset parameters. New parameter: spanwise wave angle.
H-04 — Pterosaur Soar	Source: PMC 2011 pterosaur wind tunnel tests. High CL_max with flexible membrane at low reduced frequency. Wings adapted for thermal soaring with progressive stall. MCO-1D test: 0.8 Hz frequency, 32 degree AoA, maximum amplitude. CAUTION: requires height above 30 m. Tests at near-stall AoA with stall protection active.
H-05 — Hawkmoth LEV Management	Source: Nakata et al. FEM study of hawkmoth. Spanwise amplitude gradient delays leading edge vortex breakdown near the wingtip, increasing span efficiency. MCO-1D test: outer wings run at larger amplitude than inner wings. New parameter: tip-to-root amplitude ratio.
H-06 — Negative Cascade	Source: Maybury and Lehmann JEB 2004. Hindwing leading forewing by 90 degrees extracts maximum energy from forewing downwash. MCO-1D test: reverse the cascade direction — W4 leads, W1 follows. Pure phase sign reversal, no mechanical change.

H-07 — Split-Frequency Resonance	Source: Dragonfly takeoff — alternating large and small amplitude wingbeats. W1 and W2 at base frequency, W3 and W4 at 10 percent higher. Beat frequency creates slow amplitude modulation that may prevent wake resonance lock-in in hover. CAUTION: frequency differential kept below 1.5 Hz.
H-08 — Manta Strouhal Optimisation	Source: Manta ray propulsive efficiency 89 percent at Strouhal numbers 0.2 to 0.4 — the universal optimal range across whales, bats, manta rays, and fish. MCO-1D: computes current Strouhal number and adjusts frequency and amplitude to bring it into the 0.2-0.4 range. Not a new configuration — a new fitness criterion.
H-12 — Locust Reversal Burst	Source: Dickinson clap-and-fling, locust unsteady aerodynamics. Faster stroke reversal generates a stronger leading edge vortex that the next stroke captures. MCO-1D test: command ODrive at maximum velocity for the last and first 10 percent of each stroke. Motor current monitored — abort if spike detected.

32.3 Pending Hypotheses (Require Validation Before Testing)

H-09 — Bat Downstroke Ratio	Asymmetric stroke timing: downstroke occupies 60-65 percent of cycle for more thrust time, upstroke 35-40 percent. Requires ODrive PWM duty cycle asymmetry. PENDING TC-02: validate ODrive behaviour under asymmetric commands before enabling in flight.
H-10 — Cetacean Harmonic Stroke	Add a second harmonic to the stroke waveform: sin(wt) plus 0.2 times sin(2wt plus phase). Creates a stroke that lingers at peak amplitude like a whale fluke. PENDING BENCH: non-standard ODrive velocity command needs bench testing for rope drive compatibility.
H-11 — Swift Within-Stroke Fold	W4 partially folds during the downstroke to reduce drag and extends on the upstroke to maximise lift area. Within-stroke planform change. PENDING BENCH: sweep servo speed at high frequency needs measurement to confirm feasibility.

SECTION 33 — EVOLUTIONARY AND RANDOM SEARCH

33.1 Why Evolution and Randomness

Biological evolution discovered metachronal locomotion in crustaceans, dragonflies, and cephalopods over hundreds of millions of years through random mutation and selection. Nature is constrained by evolutionary history — it can only modify what already exists. There are aerodynamically valid configurations that no animal has ever used simply because no evolutionary path led there. Layer 4 random search explores that space without constraint.

33.2 Layer 3 — Evolutionary Mutation and Crossbreeding

Once the discovery registry has accumulated successful DISC_NNNN modes — modes that have been validated in flight and promoted to the lever map — Layer 3 uses them as parents.

Mutation	Takes one successful parent and changes 1 to 4 of its parameters simultaneously by a Gaussian random amount. The mutation boldness is set at 15 percent of the parameter range — conservative enough to stay near the proven region while exploring beyond gradient search range. The mutant is tested for 5 seconds. If it outperforms the parent by 8 percent or more, it is registered as a new DISC mode and the parent-child relationship is recorded for future generations.
Crossbreeding	Takes two successful parents and produces a hybrid. Each parameter is taken from parent A, parent B, or blended between them — with 40 percent chance for each parent and 20 percent chance of a blend at a random point between the two values. The hybrid inherits the best of both parents. Hybrids that outperform both parents are registered and become eligible as parents for the next generation.
Generational compounding	There is no fixed number of generations. Every flight adds to the discovery registry. Every new discovery is immediately eligible as a parent. A successful Layer 4 random discovery from flight 3 can be crossbred with a biomimetic hypothesis validated in flight 7 to produce a hybrid in flight 12. The system compounds without limit.

33.3 Layer 4 — Random Safety-Gated Search

Layer 4 samples all 32 parameter dimensions uniformly at random within the physically permissible bounds. It has no knowledge of what has worked before, no biological inspiration, and no gradient direction to follow. It is the purest form of exploration in the system.

Most Layer 4 trials produce neutral or slightly negative results. This is expected and correct. The system runs them anyway because the ones that produce strongly positive results are genuinely novel — configurations that neither human design nor biological evolution has encountered. These are the

discoveries most likely to be qualitatively different from anything in the existing library.

A Layer 4 success is treated identically to any other discovery: registered, named DISC_NNNN, promoted to the lever map if it beats the existing best, and immediately eligible as a Layer 3 parent. The random origin of the discovery is recorded in the knowledge file but does not affect how it is used.

■ Layer 4 is the most likely source of unexpected results. The safety gates ensure these unexpected results cannot damage the aircraft — any configuration that causes an IMU spike, current increase, or altitude loss is aborted immediately and the result discarded. Only genuinely positive results survive to promotion.

SECTION 34 — KNOWLEDGE TRANSFER — WHAT IS LEARNED AND HOW IT IS SAVED

34.1 What the Prototype Learns That Can Be Kept

Everything the prototype discovers during flight is captured in a structured knowledge file saved after every post-flight sequence. This file can be given to any future MCO aircraft of any size, so it begins its first flight with the benefit of every discovery the prototype made. The knowledge is organised into four categories with different transfer rules.

34.2 Four Categories of Knowledge

Category A — Dimensionless Principles	Transfer perfectly to any aircraft at any scale. Phase offsets in degrees, AoA in degrees, amplitude in degrees, cascade direction, frequency ratios between pairs, Strouhal number targets, spanwise wave angles, advance rotation angles, amplitude gradient ratios. These describe HOW to fly a metachronal ornithopter, not how big to fly one. A validated phase relationship discovered on the MCO-1D is equally valid on a 10-kilogram aircraft or a 500-gram vehicle.
Category B — Scaled Parameters	Transfer via established physics scaling laws. Raw frequency scales as the square root of mass divided by the 1.5 power of length ratio. Stall speed scales as the square root of wing loading. The Strouhal number approach avoids most of this: target St=0.20-0.40 is universal, so new frequency is computed from the new aircraft hover speed and wingspan rather than scaled from the prototype.
Category C — Architecture	The wave library itself — all 34,908 lines — transfers in full to any new aircraft. All nine optimizer loops, four exploration layers, twelve biomimetic hypotheses, unified lever, twelve ground states, touch and go, emergency response, knowledge transfer system. New aircraft inherits the entire architecture on day one. Only the hardware constants are repopulated from the new build measurement sheet.
Category D — Failure Registry	What was tried and did not work is as important as what succeeded. The knowledge file records: backward flight in air is not possible at T/W of 1.0 (6.77 m/s squared sink rate from reversed AoA). Frequency differential above 1.5 Hz risks rope drive interference. H-04 pterosaur soar requires height above 30 m. These transfer as safety notes to every future aircraft.

34.3 The Knowledge File Format

The knowledge file is a human-readable JSON document. Anyone can open it in a text editor and read every principle the aircraft discovered — what biological source inspired it, how many observations validated it, what its Strouhal number was, and what its AoA and phase values were. It is not a binary blob.

It is a document of what the aircraft learned.

Automatic export	mco_knowledge_auto_export() runs automatically as the last step of every post-flight sequence, before disarm. Files are numbered sequentially: mco_knowledge_001.json, mco_knowledge_002.json, and so on. The last ten are kept. A mco_knowledge_latest.json symlink always points to the most recent. No pilot action required.
Manual export	Ground station command: knowledge_export Exports immediately with timestamp. knowledge_export_usb copies to a connected USB stick automatically without specifying a path.
Import on new aircraft	knowledge_import loads the file and warm-starts all four exploration layers with the prototype discoveries. Transferred principles enter the DISC registry as XFER_MCO-1D_DISC_NNNN entries. They are re-validated in flight before being promoted to the lever map. knowledge_import_usb loads directly from USB.
Boot detection	If a knowledge file is present when the wave library loads, the ground station displays: Knowledge file from MCO-1D found — N principles available. Import at pre-flight. The pilot can accept or decline at the pre-flight check.

SECTION 35 — INTER-AIRCRAFT KNOWLEDGE TRANSFER — PRACTICAL WORKFLOW

35.1 Three Transfer Methods

Method A — USB Stick (recommended for one-to-one transfer)

Step 1	After flying the prototype: plug any USB-A flash drive into the Raspberry Pi USB port.
Step 2	Ground station command: knowledge_export_usb The file copies to the stick automatically.
Step 3	Unplug the stick and plug it into the new aircraft Pi.
Step 4	Ground station command: knowledge_import_usb If the new aircraft has a different size, add scaling parameters: mass in kg and wingspan in metres.
Step 5	Done. The knowledge is loaded and ready for pre-flight.

Method B — Ground Station Laptop Hub (recommended for a fleet)

The ground station laptop already connects to whichever aircraft is powered on. The mco_sync.py script uses this to move knowledge files between sessions without USB.

Setup (once)	Generate an SSH key pair and copy the public key to both Pis. The sync script uses passwordless SSH. Commands shown in the sync script header.
After prototype session	python3 mco_sync.py pull mco-1d.local Pulls all knowledge files to the laptop knowledge folder.
Before new aircraft session	python3 mco_sync.py push mco-2.local --mass 6.5 --span 8.2 Pushes the latest knowledge file to the new aircraft Pi. The script shows a summary of what is being transferred before the transfer happens.
One-step transfer	python3 mco_sync.py transfer mco-2.local --source mco-1d.local Pulls from the prototype and immediately pushes to the new aircraft in a single command.
Status check	python3 mco_sync.py status Shows all knowledge files on the laptop with dates, sizes, and principle counts.

Method C — SD Card Clone (for geometrically identical aircraft only)

Clone the entire SD card from the prototype to the new aircraft using standard Pi imaging tools. The new aircraft receives the complete state: wave library, all discoveries, all tuned parameters. Then update the hardware constants for the new aircraft geometry and re-run the commissioning tests. This method is only

appropriate when the two aircraft are physically identical — same wingspan, same mass, same motor and servo specification.

■ If using SD card clone on a different-sized aircraft, the hardware constants MUST be updated before any flight. Operating with the wrong stall speed, wrong geometry, or wrong sector wheel limits is a safety risk.

35.2 Re-Validation on the New Aircraft

Transferred principles are not immediately promoted to the live lever map. They enter the discovery registry as XFER_ entries flagged for re-validation. The four-layer exploration system tests each one during normal flight using the same 5-second trial with full safety gating. Principles that perform well on the new aircraft get confirmed and promoted. Principles that do not perform — because the new aircraft geometry makes them suboptimal — are discarded after three failed trials. The new aircraft reaches validated performance in a fraction of the flights it would take starting from zero.

SECTION 36 — UPDATED STATISTICS AND PENDING ITEMS

36.1 Software Statistics — Revision 6

Wave Library	34,908 lines (1.34 MB source)
Ground Station Client	588 lines
Panel Application	435 lines
Knowledge Sync Script	229 lines (mco_sync.py)
Optimizer loops	9 (Loops 1-8 tactical, Loop 9 meta)
Exploration layers	4 (gradient, biomimetic, evolutionary, random)
Biomimetic hypotheses	12 defined (9 active, 3 pending TC/bench)
Discovery dimensions	32 per-wing parameters
Ground states	12 unified states with hysteresis
Knowledge file format	JSON version 1.0, human-readable
Tick stack depth	31 layers
Tick time (Pi 4)	~2.6 ms per tick (13 percent of 20 ms budget)
RAM usage	~3.0 MB peak

36.1a Motor Specification — Confirmed

The GoolRC 5010-14 brushless motor specification is confirmed from the manufacturer datasheet. No field measurement is required. The 12N14P designation is standard across all units of this model.

Model	GoolRC 5010-14
KV rating	360 KV
Configuration	12N14P — 12 stator slots, 14 magnetic poles
Pole pairs	7 (14 poles ÷ 2 = 7 pole pairs)
Stator coils	12
Rotor magnets	14 permanent magnets
Quantity	8 motors (one per wing)
ODrive setting	axis.motor.config.pole_pairs = 7
Wave library	MOTOR_POLE_PAIRS = 7 (confirmed constant)

36.2 Pending Items — Not Yet Implemented

dart_left / dart_right	Gesture trigger design pending operational definition. Parameters and logic exist. Safety-gated entry combo must be unambiguous and unreachable by accident.
inclined_hover	Requires TC-02 body-pitch thrust measurement at 52 degrees. Call inclined_hover_calibrate() after TC-02. Gesture will be a deliberate triple-input combination.
H-09 — Asymmetric downstroke ratio	Requires ODrive PWM duty cycle asymmetry validation. Bench test first, then enable after TC-02.
H-10 — Cetacean harmonic stroke waveform	Non-standard ODrive velocity command. Bench test for rope drive compatibility before flight.
H-11 — Swift within-stroke W4 fold	Sweep servo speed at high frequency must be measured. Feasibility depends on servo response time vs stroke period.
Stroke plane controller	Post TC-02/TC-03: generative system taking a desired world-frame force vector and synthesising complete wing commands for all 8 wings. Unlocks backward flight, sideways translation, knife-edge split stroke. Framework ready. Needs measured CT-J data first.
ODrive vel_limit	Set to 150 rad/s and bench-test frequency ceiling before first flight. Currently at conservative default. Motor pole pairs are CONFIRMED — GoolRC 5010-14, 12N14P, 7 pole pairs. No field verification needed.
Build Measurement Sheet	Fill MCO-1D_Build_Measurement_Sheet.pdf from physical hardware measurements. Required before TC-01.

36.3 The Knowledge Transfer Promise

Every flight the MCO-1D makes adds to a knowledge base that will be given to every future MCO aircraft. The prototype is not just testing one aircraft — it is building the foundational knowledge for an entire lineage of metachronal ornithopters. Dimensionless principles transfer at any scale. Scaling laws convert dimensional parameters. The architecture transfers in full. The failure registry prevents every future aircraft from re-discovering the same limits the hard way.

The biomimetic hypothesis library will grow. Every new biological research paper on dragonfly, bat, pterosaur, manta ray, or any other flying, swimming, or terrestrial locomotion system can be translated into a new hypothesis class and injected into the exploration system. The aircraft does not need to be rebuilt to benefit from new biological insights — they are added as software and tested in the next flight.

SECTION 37 — COMPLETE SOFTWARE STACK AND INSTALLATION

37.1 The Complete File Inventory

The MCO-1D software consists of nine Python files, two shell scripts, two systemd service files, and the ground station tools. Every file has a single clearly defined purpose. No file should be missing before attempting commissioning.

mco_wave_library.py (34,964 lines)	The flight control core. Nine optimizer loops, four exploration layers, twelve biomimetic hypotheses, thirty-one tick stack layers, all ground operations, knowledge transfer system. Runs on the Pi.
mco_main.py (307 lines)	The entry point. The file you actually run. Creates the 50 Hz control loop, wires all subsystems together, handles graceful shutdown and auto-export. Run directly: python3 mco_main.py
mco_mavlink.py (290 lines)	MAVLink sensor bridge. Background thread reads Pixhawk TELEM2 at 921600 baud, translates MAVLink messages into the sensor dictionary the wave library expects at 50 Hz. Handles ATTITUDE, VFR_HUD, GLOBAL_POSITION_INT, RANGEFINDER, RC_CHANNELS.
mco_gs_server.py (193 lines)	TCP socket server on the Pi. Accepts connections from the ground station laptop and the touchscreen panel. Broadcasts telemetry at 10 Hz to all clients. Queues incoming commands for the flight loop. Port 5757.
mco_logger.py (185 lines)	Non-blocking 50 Hz flight data logger. Writes CSV to /home/pi/logs/mco_flight_NNN.csv. Logs all sensor values, wing amplitudes, mode, discovery status, optimizer layer. Essential for TC data collection.
mco_odrive_setup.py (187 lines)	ODrive first-time configuration. Run once after mechanical assembly. Sets pole pairs, motor type, current limits, velocity limit (150 rad/s), CAN node IDs. Run with --calibrate for full motor and encoder calibration. Run with --verify to check current settings at any time.
mco_panel.py (435 lines)	Touchscreen panel. Four buttons: PRE-FLIGHT, POST-FLIGHT, STORM STOW, EMERGENCY STOP. Connects to gs_server via same socket as laptop GS.
mco_ground_station.py (588 lines)	Ground station client. Runs on the laptop. Connects to mco_gs_server.py on the Pi. Sends commands, receives telemetry.
mco_sync.py (229 lines)	Knowledge transfer sync tool. Runs on laptop. pull, push, transfer, status commands for moving knowledge files between aircraft via SSH/rsync.
mco_can_setup.sh	Shell script. Brings up the CAN-FD interface on the Pi. Must run before any ODrive communication. Called from /etc/rc.local on boot.

mco_install.sh	One-time installation script. Run once on a fresh Raspberry Pi OS. Installs all Python packages, enables pigpio, creates directories, installs systemd services, configures CAN for boot.
mco-flight.service	Systemd unit file. After installation, the flight computer starts automatically on every boot. Restarts automatically if it crashes. Requires pigpiod service.
mco-panel.service	Systemd unit file. Touchscreen panel starts automatically after the flight service. Uses framebuffer (no desktop environment needed).

37.2 Installation Sequence (Once Per Aircraft)

Run this sequence once on a fresh Raspberry Pi OS Bookworm installation before any commissioning tests. All steps are covered by mco_install.sh which can be run as a single command after copying the files.

Step 1 — Copy files	Copy all MCO software files to /home/pi/ on the Pi. Use SCP, USB stick, or direct SD card write.
Step 2 — Run install script	sudo bash /home/pi/mco_install.sh Installs: pymavlink, odrive, python-can, pigpio, pygame. Enables pigpiod. Creates /home/pi/logs/ and knowledge/. Installs and enables systemd services. Adds CAN setup to rc.local.
Step 3 — Configure CAN hardware	Ensure MCP2518FD or compatible CAN-FD hat is fitted to the Pi SPI header. Verify device tree overlay is enabled in /boot/config.txt for your specific hat.
Step 4 — Configure ODrives	With CAN connected and ODrives powered: python3 /home/pi/mco_odrive_setup.py --calibrate This configures all four ODrives and runs motor and encoder calibration. Takes approximately 5 minutes. Run once only. Results are saved to ODrive flash.
Step 5 — Configure Pixhawk	Connect Pixhawk TELEM2 to Pi /dev/ttyAMA0. Set TELEM2 baud rate to 921600 in Pixhawk parameters. Verify RC channel mapping matches mco_mavlink.py defaults: CH1=roll, CH2=pitch, CH3=throttle/lever, CH4=yaw.
Step 6 — Test without motors	python3 /home/pi/mco_main.py --sim Verifies the full software stack without hardware output. Ground station should connect and receive telemetry.
Step 7 — Reboot	sudo reboot Flight computer and panel start automatically. No SSH or manual launch needed for normal operations.

37.3 The 50 Hz Control Loop

Every 20 milliseconds, the following sequence runs:

1. Get sensors	mco_mavlink.py background thread has read MAVLink continuously. get_sensors() returns the current dict without waiting — always fresh data available.
2. Process GS commands	mco_gs_server.py queues incoming commands. The flight loop processes them here, calls _gs_execute_cmd(), and queues the acknowledgement.
3. Flight control tick	MCOFlightController.tick(sensors) runs all 31 layers of the tick stack. Takes approximately 2.6 ms on Pi 4. Returns wing commands, frequency, mode information.
4. Apply to hardware	Servo PWM commands via pigpio. ODrive velocity commands via CAN-FD. Both happen in parallel, non-blocking.
5. Log	mco_logger.py puts the record into a queue. Background thread writes to CSV. Never blocks the control loop.
6. Telemetry	Every 5 ticks (10 Hz): pack telemetry dict and give it to mco_gs_server.py for broadcast. Non-blocking — server broadcasts on its own schedule.
7. Sleep	Sleep for remainder of 20ms tick period. If tick took longer than 20ms: proceed immediately without sleeping (no tick skipping).

37.4 Flight Data Logs

Every flight session creates a new numbered CSV file at /home/pi/logs/mco_flight_NNN.csv. The last 50 sessions are kept. Each file contains 50 records per second with the following fields:

timestamp_s, elapsed_s	Absolute and relative time in seconds.
ground_state	Current twelve-state ground machine state.
h_agl_m, V_ms	Height above ground and airspeed.
pitch_deg, roll_deg, heading_deg	Aircraft attitude from IMU.
sink_rate_ms, accel_z_g	Vertical velocity and vertical acceleration.
freq_hz, lever_pct, Ps_measured	Wing frequency, lever position, specific excess power.
bus_voltage_v, motor_I_A	Battery voltage and motor current.
w1_port_amp through w4_stbd_amp	Flap amplitude for each of the 8 wings individually.
mode_id, ground_ops_mode	Active flight mode and ground operations state.
discovery_active, n_disc_promoted	Discovery system status and promotion count.
l9_active, exploration_layer	Meta-optimizer status and active exploration layer.

■ Log files are essential for TC-01, TC-02, and TC-03 data collection. Always review the log after any commissioning test before proceeding to the next phase. Copy logs to the laptop with: scp

```
pi@mco-1d.local:/home/pi/logs/*.csv ./logs/
```

37.5 Updated Complete Software Statistics — Revision 7

mco_wave_library.py	34,964 lines — flight control core
mco_main.py	307 lines — entry point and flight loop
mco_mavlink.py	290 lines — Pixhawk sensor bridge
mco_gs_server.py	193 lines — socket server on Pi
mco_logger.py	185 lines — 50 Hz flight data logger
mco_odrive_setup.py	187 lines — ODrive configuration
mco_panel.py	435 lines — touchscreen panel
mco_ground_station.py	588 lines — laptop GS client
mco_sync.py	229 lines — knowledge transfer
Shell and config files	4 files — install, CAN, systemd
TOTAL	37,378 lines across 9 Python files
Control loop rate	50 Hz (20 ms period)
Telemetry rate	10 Hz to ground station
Log rate	50 Hz CSV to SD card
Tick time (Pi 4)	~2.6 ms (13 percent of budget)

SECTION 38 — THREE-TIER OBJECTIVE SYSTEM

38.1 The Problem with a Single Objective

The optimizer previously used one metric for every situation: specific excess power (Ps). Ps measures whether the aircraft has more thrust than drag at the current speed. This is the correct goal for exactly one flight phase — takeoff, sprint, and emergency. For every other phase it is either irrelevant or actively counterproductive.

At efficient cruise the pilot wants level flight on minimum power. Maximising Ps at cruise means burning the most battery to generate excess thrust nobody needs. At approach, maximising Ps fights the descent. At SAR loiter, Ps says nothing about whether the sensor platform is stable enough to be useful. The optimizer needs to know what success means in each situation.

38.2 The Three Tiers

Tier 1 — Safety Envelope	The hard outer boundary that is never crossed. Stall margins, structural limits, current limits, ground clearance. Already built as the safety gating system. Never moves inward.
Tier 2 — Mission Phase Objectives	Within the safety envelope, optimise for the current task. Each flight phase has its own scoring function. The optimizer pursues the correct goal automatically based on the active ground state. 95 percent of optimizer time is here.
Tier 3 — Envelope Expansion	A dedicated module that maps the demonstrated safe boundary and takes deliberate conservative steps beyond it. 5 percent of exploration time in suitable conditions. Grows the aircraft capabilities incrementally over many flights without any single dramatic change.

38.3 Phase-Indexed Scoring Functions

Seven scoring functions, one per mission context. Each returns a normalised score where higher is always better, so the discovery and exploration systems need no changes — they evaluate trials against the correct metric automatically.

Takeoff, sprint, emergency → Ps directly	These are the phases where excess power genuinely matters. The optimizer maximises specific excess power exactly as before.
Transit → Range per watt-hour	Speed divided by power consumed, with a factor penalising configurations that sacrifice forward progress for Ps. Efficient cruise at 9 m/s on 80 watts scores much better than burning 160 watts to generate unneeded climb capability.
Cruise → Minimum power for level flight	Inverse of power consumed, with penalty for altitude loss. The optimizer finds the minimum-power configuration that holds level flight — efficient cruise.

SAR loiter → Endurance times stability	60 percent endurance (inverse of power) plus 40 percent stability (low IMU RMS for sensor platform). A slightly less efficient but smoother configuration outscore a more powerful but vibration-prone one.
Approach and flare → Precision	Proximity to a gentle controlled descent: slight negative Ps at 0.3 m/s sink rate. Configurations that hold this precisely and smoothly score highest.
Hover → Minimum power at held altitude	Same as cruise scoring but in the hover phase context. Find the configuration that holds altitude on least power.

38.4 The Merged Objective

The phase-indexed objectives and envelope expansion work at different timescales and are complementary rather than competing. Mission phase optimisation happens every flight, continuously refining the best known configuration for each task. Envelope expansion happens incrementally over many flights, discovering new capabilities. A larger demonstrated envelope gives the phase-indexed optimizer more parameter space to work in. Better phase performance reveals the edges of the envelope more clearly.

■ Ground station command: objective_status prints the current phase, active scoring metric, and most recent score for the current lever position.

SECTION 39 — FLIGHT ENVELOPE STATE AND EXPANSION

39.1 The Demonstrated Flight Envelope

The flight envelope is the set of all conditions the aircraft can safely sustain and depart from. The MCO-1D tracks the demonstrated boundary across eight axes. "Demonstrated" means the aircraft has actually reached and departed that value safely in flight — not a theoretical limit, but an empirical one.

Airspeed	Maximum forward speed demonstrated safely
Altitude AGL	Maximum height above ground demonstrated
Bank angle	Maximum roll angle demonstrated
Climb rate	Maximum sustained climb rate demonstrated
Sink rate	Maximum controlled descent rate demonstrated
Turn rate	Maximum yaw rate demonstrated
Gust intensity	Maximum IMU RMS survived without abort
Crosswind	Maximum crosswind component operated in

Each axis records its demonstrated maximum, how many times that extreme has been confirmed across multiple flights, and the last time it was observed. The envelope state persists in /home/pi/knowledge/mco_envelope.json and transfers to future aircraft in the knowledge file.

39.2 Continuous Observation

Every sensor tick during normal flight, envelope_observe() checks whether any current sensor reading exceeds 95 percent of the demonstrated value on any axis. If it does, the demonstrated boundary is updated and the confirmation count incremented. The envelope fills in passively during every flight without any dedicated trials.

39.3 Expansion Probes

Five percent of exploration time is allocated to deliberate expansion probes. The system selects the axis that has been explored least recently with the fewest confirming observations — the coldest boundary dimension. It then commands a target 8 percent beyond the current demonstrated value and monitors for 8 seconds.

Probe conditions	Altitude above 30 m. Turbulence below 0.10. Speed above 1.3 times stall. Calm air only. Not during critical phases.
Step size	8 percent beyond demonstrated value. Maximum 12 percent per step regardless of history. Conservative by design — one small step at a time.

Success criterion	Aircraft reaches within 10 percent of target value and all abort conditions remain clear for 8 seconds. Demonstrated boundary updated. Confirmation count incremented.
Failure and retreat	Any abort condition (IMU spike, altitude loss, stall risk) terminates the probe immediately. Step size at that axis tightens by 15 percent — the system becomes more conservative at that specific boundary until more data is available.
Compounding over flights	Each successful probe extends the demonstrated envelope by one data point. Over many flights the boundary fills in from all directions. The pilot notices the aircraft handling increasingly extreme situations with confidence over time.

■ Ground station command: envelope_status prints the current demonstrated boundary on all eight axes with confirmation counts.

SECTION 40 — HUMAN HYPOTHESIS INJECTION

40.1 The Problem Being Solved

The four-layer exploration system generates ideas autonomously. But the people operating the aircraft — pilots, engineers, researchers — also have ideas. A researcher reads a paper at midnight about butterfly wing kinematics. A pilot notices something unexpected during a flight. An engineer has a physical intuition about a configuration worth testing. These insights should be able to enter the optimizer without requiring Python programming, SSH access, or source code changes.

40.2 Four Levels of Input

Level 1 — Named hypothesis	Request a built-in hypothesis by name: "Try H-01 at the climb lever position." Already works via ground station command. No file needed.
Level 2 — Explicit parameters	Specify exact values: AoA 28 degrees, cascade phase 45 degrees, freq 2.2 Hz at the cruise lever. Write a JSON file in five minutes. No programming knowledge required.
Level 3 — Described principle	Describe a biological observation: "Bumblebees use a figure-8 stroke path that increases lift by 30 percent. Try encoding this as higher sweep amplitude combined with reversed cascade." Requires translation into parameters — done with Claude's help at the laptop before flight.
Level 4 — Freeform observation	"Monarch butterflies glide between wingbeats with wings fully extended at high dihedral. Try it." Same as Level 3 — describe it, Claude maps it to parameters, inject the resulting JSON. No internet needed on the aircraft.

40.3 The Hypothesis JSON Format

A minimal valid hypothesis file has six fields and can be written in five minutes on any device — laptop, phone, or tablet. Safety limits are enforced at both the authoring tool and the wave library. An out-of-range value is rejected before it ever reaches the aircraft.

name	Short description of the hypothesis. Appears in all reports and the knowledge file.
source	Where the idea came from — paper citation, biological observation, physical intuition. Preserved permanently.
lever_pct	Where in the flight regime to test this. 0-100%, or zone name: takeoff, climb, cruise, trim, glide, approach, hover.
params	The parameter values to test. Only specify what you want to change from defaults. Any subset of the 32 dimensions.

notes	Optional free-text description of the idea and what you expect to observe.
safety (optional)	Override minimum test altitude (default 15 m) and maximum number of trials (default 3). The system's minimum altitude cannot be reduced below 15 m.

40.4 Four Injection Paths

USB stick	Name the file hypothesis_NNN.json and place it in the root of a USB stick. Plug into the Pi. The library scans for files at boot and at pre-flight. Ground station confirms: H-USER-001 queued.
Ground station command	inject_hypothesis_file /path/to/file.json Or: inject_hypothesis followed by the JSON string directly. Works mid-session without rebooting.
Hypothesis inbox	Drop files into /home/pi/hypothesis_inbox/ on the Pi via SCP or USB. Scanned automatically at boot and pre-flight.
Hypothesis tool (direct)	python3 mco_hypothesis_tool.py --inject --host mco-1d.local Sends directly to the aircraft socket without a file. Fastest path when the laptop is connected.

40.5 What Happens After Injection

Queued	Hypothesis enters Layer 2 exploration queue as H-USER-NNN. Tested in flight using the same 5-second trial system as all built-in hypotheses. Full safety gating applies.
Trials	Up to the specified maximum trials (default 3). After each trial the ground station reports the score versus the baseline at that lever position.
Promotion	If the mean score beats the existing best by 8 percent or more: promoted to the live lever map. The map immediately uses the new configuration at that lever position on subsequent flights.
Knowledge transfer	Promoted user hypotheses enter the discovery registry as USER_NNN entries. They transfer to future aircraft in the knowledge file exactly like any other discovery. A good idea injected today compounds into every future aircraft in the lineage.
Not promoted	If trials do not beat the baseline: hypothesis is marked complete and the result is logged. The idea was tested honestly and the aircraft found its existing configuration better. This is useful information too.

40.6 The mco_hypothesis_tool.py

A laptop tool for authoring hypothesis files without writing JSON by hand. Three modes:

Interactive mode	python3 mco_hypothesis_tool.py Guided step-by-step prompts. Accepts zone names for lever position. Shows allowed ranges for each parameter. Saves numbered JSON file to current directory.
Quick mode	python3 mco_hypothesis_tool.py --name "High dihedral hover" --source "Observation" --lever hover --params "global_aoa_deg=28,freq_hz=1.5" Single command, no prompts.
Direct inject	Add --inject --host mco-1d.local to either mode. Sends the hypothesis directly to the aircraft socket. No file, no USB, no pre-flight scan needed.
List	python3 mco_hypothesis_tool.py --list Shows all saved hypothesis files with names and lever positions.

■ Hypothesis files written on phones or tablets must use valid JSON format. The hypothesis tool validates this automatically. If writing by hand, use a JSON validator before copying to USB.

SECTION 41 — UPDATED STATISTICS AND PENDING ITEMS — REVISION 9

41.1 Complete Software File Inventory

mco_wave_library.py (35,810 lines)	Flight control core. Nine optimizer loops, four exploration layers, twelve biomimetic hypotheses, three-tier objective system, envelope tracker, human hypothesis injection.
mco_main.py (307 lines)	Entry point. 50 Hz flight loop.
mco_mavlink.py (290 lines)	Pixhawk MAVLink sensor bridge.
mco_gs_server.py (193 lines)	TCP ground station server on Pi.
mco_logger.py (185 lines)	50 Hz CSV flight data logger.
mco_odrive_setup.py (187 lines)	ODrive configuration (pole pairs confirmed: 7).
mco_panel.py (435 lines)	Touchscreen panel — four buttons.
mco_ground_station.py (588 lines)	Laptop ground station client.
mco_sync.py (229 lines)	Knowledge transfer sync between aircraft.
mco_hypothesis_tool.py (287 lines)	Laptop tool for authoring and injecting hypotheses.
Shell and config files	mco_install.sh, mco_can_setup.sh, mco-flight.service, mco-panel.service.
TOTAL	38,511 lines across 10 Python files.

41.2 Complete Ground Station Commands

arm	Arm ODrives and enable flight mode
disarm	Safe shutdown
emergency_stop	Instant wing stop
ground_ops_taxi	Enter taxi mode
ground_ops_cooldown	Start 8-minute cooldown
ground_ops_aft_stow	Fold wings to aft storage
storm_stow	Storm parking configuration
preflight_start	Run all three startup check phases
postflight_start	Start post-flight sequence

l9_status	Print Loop 9 meta-optimizer status
l9_freeze	Freeze meta-optimizer parameters
exploration_status	Print four-layer exploration status
envelope_status	Print demonstrated flight envelope
knowledge_export	Export knowledge file now
knowledge_export_usb	Copy knowledge file to USB
knowledge_import_usb	Load knowledge file from USB
knowledge_status	Show current knowledge state
inject_hypothesis	Inject hypothesis JSON string
inject_hypothesis_file	Inject hypothesis from file path
load_hypotheses_usb	Scan USB for hypothesis files
user_hypothesis_status	Show all injected user hypotheses

41.3 Pending Items — Not Yet Implemented

dart_left / dart_right	Gesture trigger design pending operational definition.
inclined_hover	Requires TC-02 body-pitch T/W measurement at 52 degrees.
H-09 — Asymmetric downstroke ratio	Requires ODrive PWM asymmetry bench validation post TC-02.
H-10 — Cetacean harmonic stroke	Non-standard ODrive command — bench test required.
H-11 — Swift within-stroke W4 fold	Sweep servo speed at frequency needs measurement.
Stroke plane controller	Post TC-02/TC-03 only. Needs measured CT-J curve.
ODrive vel_limit	Set 150 rad/s and bench-test frequency ceiling. Pole pairs confirmed at 7 — no field verification needed.
Build Measurement Sheet	Fill from physical hardware before TC-01.
HX711 load cell reader	mco_loadcell.py needed for TC-02 static thrust measurement.
RC channel mapping	Verify CH1=roll, CH2=pitch, CH3=lever, CH4=yaw matches radio.

41.4 What the Optimizer Now Pursues

The complete objective hierarchy as of Revision 9:

Takeoff, sprint, emergency	Specific excess power — maximum thrust available.
Transit and cruise	Range per watt-hour — efficient forward progress.
Level flight	Minimum power for altitude hold.

SAR loiter and search	Endurance weighted with sensor platform stability.
Approach and flare	Precision — controlled descent at target sink rate.
Envelope expansion	5 percent of exploration time probing beyond demonstrated limits, 8 percent step, full sensor gating, persists across flights.
Human hypotheses	Injected ideas tested at the same standard as all other discoveries. Good ones promoted and transferred permanently.